
ЧАСТЬ III

ИНСТРУМЕНТЫ ДЛЯ РАЗРАБОТЧИКОВ КЛАССОВ

В этой части...

Глава 13. Управление копированием

Глава 14. Перегрузка операторов и преобразований

Глава 15. Объектно-ориентированное программирование

Глава 16. Шаблоны и обобщенное программирование

Классы — основная концепция языка C++. Подробное рассмотрение определения классов начато в главе 7. Она затрагивает такие фундаментальные для классов темы, как область видимости класса, сокрытие данных и конструкторы. Она познакомила также с важнейшими средствами класса: функциями-членами, неявным указателем `this`, дружественными отношениями, а также членами `const`, `static` и `mutable`. В этой части дополним тему классов, рассмотрев управление копированием, перегрузку операторов, наследование и шаблоны.

Как уже упоминалось, в классах C++ определяются конструкторы, контролирующие происходящее при инициализации объектов класса. Классы контролируют также то, что происходит при копировании, присвоении, перемещении и удалении объектов. В этом отношении язык C++ отличается от других языков, большинство из которых не позволяет разработчикам классов контролировать эти операции. В главе 13 будут рассмотрены эти темы, а также две важные концепции, введенные новым стандартом: ссылки на `r`-значения и операции перемещения.

Глава 14 посвящена перегрузке операторов, позволяющей использовать операнды типа классов со встроенными операторами. Перегрузка оператора — это один из способов, которым язык C++ позволяет создавать новые типы, столь же интуитивно понятные в использовании, как и встроенные типы.

Среди доступных для перегрузки операторов класса есть оператор вызова функции. Объекты таких классов можно “вызвать” точно так же, как если бы

они были функциями. Рассмотрим также новые библиотечные средства, облегчающие использование различных типов вызываемых объектов единообразным способом.

Эта глава завершается рассмотрением еще одного специального вида функций-членов класса — операторов преобразования. Эти операторы определяют неявные преобразования из объектов типа класса. Компилятор применяет эти преобразования в тех же контекстах (и по тем же причинам), что и преобразования встроенных типов.

Последние две главы этой части посвящены поддержке языком C++ объектно-ориентированного и обобщенного программирования.

Глава 15 рассматривает наследование и динамическое связывание. Наряду с абстракцией данных наследование и динамическое связывание — это основы объектно-ориентированного программирования. Наследование облегчает определение связанных типов, а динамическое связывание позволяет писать независимый от типов код, способный игнорировать различия между типами, которые связаны наследованием.

Глава 16 посвящена шаблонам классов и функций. Шаблоны позволяют писать обобщенные классы и функции, которые не зависят от типов. Новый стандарт ввел множество новых средств, связанных с шаблонами: шаблоны с переменным количеством аргументов, псевдонимы типов шаблона и новые способы контроля создания экземпляра.

Создание собственных объектно-ориентированных или обобщенных типов требует довольно хорошего понимания языка C++. К счастью, для их использования это не обязательно. Например, стандартная библиотека интенсивно использует средства, которые рассматриваются только в главах 15 и 16, но библиотечные типы и алгоритмы использовались уже с самого начала книги, даже без объяснения их реализации.

Поэтому читатели должны понимать, что часть III посвящена довольно сложным средствам. Написание шаблонов и объектно-ориентированных классов требует хорошего понимания основ языка C++ и глубокого знания того, как определяют базовые классы.

ГЛАВА 13

УПРАВЛЕНИЕ КОПИРОВАНИЕМ

В этой главе...

13.1. Копирование, присвоение и удаление	630
13.2. Управление копированием и ресурсами	647
13.3. Функция <code>swap()</code>	654
13.4. Пример управления копированием	658
13.5. Классы, управляющие динамической памятью	664
13.6. Перемещение объектов	673
Резюме	695
Термины	695

Как упоминалось в главе 7, каждый класс является определением нового типа и операций, которые можно выполнять с объектами этого типа. В этой главе упоминалось также о том, что классы могут определять конструкторы, которые контролируют происходящее при создании объектов данного типа.

В этой главе мы изучим то, как классы могут контролировать происходящее при копировании, присвоении, перемещении и удалении объектов данного типа. Для этого классы имеют специальные функции-члены: конструктор копий, конструктор перемещения, оператор присвоения копии, оператор присваивания при перемещении и деструктор.

При определении класса разработчик (явно или неявно) определяет происходящее при копировании, перемещении, присвоении и удалении объектов данного класса. Класс контролирует эти операции, определяя пять специальных функций-членов: *конструктор копий* (copy constructor), *оператор присвоения копии* (copy-assignment operator), *конструктор перемещения* (move constructor), *оператор присваивания при перемещении* (move-assignment operator) и *деструктор* (destructor). Конструкторы копирования и перемещения определяют происходящее при инициализации объекта данными из другого объекта того же типа. Операторы копирования и присваивания при перемещении определяют происходящее при присвоении объекта данного класса другому объекту того же класса. Деструктор определяет происходящее в момент,

когда объект данного типа прекращает существование. Все эти операции вместе мы будем называть *управлением копированием* (copy control).

Если класс определяет не все функции-члены управления копированием, компилятор сам определит недостающие. В результате многие классы могут не определять управление копированием (см. раздел 7.1.5, стр. 347). Но некоторые классы не могут полагаться на заданные по умолчанию определения. Зачастую наиболее трудная часть реализации операций управления копированием — это принятие решения об их необходимости.



Управление копированием — это важнейшая часть определения любого класса C++. У начинающих программистов C++ зачастую возникают затруднения при необходимости определения действий, происходящих при копировании, перемещении, присвоении и удалении объектов. Это затруднение обусловлено тем, что задавать их не обязательно, компилятор вполне может создать их сам, хотя результат этих действий может быть не совсем таким, как хотелось бы.

13.1. Копирование, присвоение и удаление

Начнем с наиболее простых операций: конструктора копий, оператора присвоения копии и деструктора. Операции перемещения (введенные новым стандартом) рассматриваются в разделе 13.6 (стр. 672).



13.1.1. Конструктор копий

Если первый параметр конструктора — ссылка на тип класса, а все дополнительные параметры имеют значения по умолчанию, то это конструктор копий:

```
class Foo {
public:
    Foo(); // стандартный конструктор
    Foo(const Foo&); // конструктор копий
    // ...
};
```

По причинам, которые будут описаны ниже, первый параметр должен иметь ссылочный тип. Он почти всегда является ссылкой на константу, хотя вполне можно определить конструктор копий, получающий ссылку на не константу. При некоторых обстоятельствах конструктор копий используется неявно. Следовательно, конструктор копий обычно не следует объявлять как `explicit` (см. раздел 7.5.4, стр. 383).

Синтезируемый конструктор копий

Если конструктор копий не определен для класса явно, компилятор синтезирует его сам. В отличие от синтезируемого стандартного конструктора (см. раздел 7.1.4, стр. 342), конструктор копий синтезируется, даже если определены другие конструкторы.

Как будет продемонстрировано в разделе 13.1.6 (стр. 644), *синтезируемый конструктор копий* (synthesized copy constructor) некоторых классов препятствует копированию объектов этого типа. В противном случае синтезируемый конструктор копий осуществляет *почленное копирование* (memberwise copy) членов своего аргумента в создаваемый объект (см. раздел 7.1.5, стр. 347). Компилятор по очереди копирует каждую нестатическую переменную-член заданного объекта в создаваемый.

Способ копирования каждой переменной-члена определяет ее тип: для типов класса применяется конструктор копий этого класса, а члены встроенного типа копируются непосредственно. Хотя нельзя непосредственно скопировать массив (см. раздел 3.5.1 стр. 162), синтезируемый конструктор копий копирует члены типа массива поэлементно. Элементы типа класса копируются с использованием конструкторов копий элементов.

Например, синтезируемый конструктор копий для класса `Sales_data` эквивалентен следующему:

```
class Sales_data {
public:
    // другие члены и конструкторы как прежде
    // объявление, эквивалентное синтезируемому конструктору копий
    Sales_data(const Sales_data&);
private:
    std::string bookNo;
    int units_sold = 0;
    double revenue = 0.0;
};
// эквивалент конструктора копий, синтезированный для класса Sales_data
Sales_data::Sales_data(const Sales_data &orig):
    bookNo(orig.bookNo),           // использование конструктора копий string
    units_sold(orig.units_sold), // копирует orig.units_sold
    revenue(orig.revenue)         // копирует orig.revenue
    {                             // пустое тело
```

Инициализация копией

Теперь можно полностью рассмотреть различия между прямой инициализацией и инициализацией копией (см. раздел 3.2.1, стр. 126):

```
string dots(10, '.');           // прямая инициализация
string s(dots);                 // прямая инициализация
string s2 = dots;               // инициализация копией
string null_book = "9-999-99999-9"; // инициализация копией
string nines = string(100, '9'); // инициализация копией
```

При прямой инициализации от компилятора требуется использовать обычный выбор функции (см. раздел 6.4, стр. 305) для подбора конструктора, наилучшим образом соответствующего предоставленным аргументам. Когда используется *инициализация копией* (copy initialization), от компилятора требуется скопировать правый операнд в создаваемый объект, осуществляя преобразования в случае необходимости (см. раздел 7.5.4, стр. 380).

Инициализация копией обычно использует конструктор копий. Но, как будет продемонстрировано в разделе 13.6.2 (стр. 676), если у класса есть конструктор перемещения, то инициализация копией иногда использует конструктор перемещения вместо конструктора копий, а пока достаточно знать, что при инициализации копией требуется либо конструктор копий, либо конструктор перемещения.

Инициализация копией осуществляется не только при определении переменных с использованием оператора `=`, но и при:

- передаче объекта как аргумента параметру не ссылочного типа;
- возвращении объекта из функции с не ссылочным типом возвращаемого значения;
- инициализации списком в скобках элементов массива или членов агрегатного класса (см. раздел 7.5.5, стр. 384)

Некоторые классы используют также инициализацию копией для резервируемых объектов. Например, библиотечные контейнеры инициализируют копией свои элементы при инициализации контейнера, либо при вызове функции `insert()` или функции `push()` элемента (см. раздел 9.3.1, стр. 439). Элементы, созданные функцией `emplace()`, напротив, отличаются прямой инициализацией (см. раздел 9.3.1, стр. 444).

Параметры и возвращаемые значения

Во время вызова функции с параметрами не ссылочного типа осуществляется инициализация копией (см. раздел 6.2.1, стр. 277). Точно так же, когда у функции не ссылочный тип возвращаемого значения, возвращаемое значение используется в точке вызова для инициализации копией результата оператора вызова (см. раздел 6.3.2, стр. 294).

Тот факт, что конструктор копий используется для инициализации не ссылочных параметров типа класса, объясняет, почему собственный параметр конструктора копий должен быть ссылкой. Если бы этот параметр не был ссылкой, то вызов не был бы успешным — при вызове конструктора копий должен быть использован конструктор копий для копирования аргумента, но для копирования аргумента следует вызвать конструктор копий и так далее до бесконечности.

Ограничения на инициализацию копией

Как уже упоминалось, используется ли инициализация копией или прямая инициализация, если используется инициализатор, то потребуется преобразование в явный конструктор (см. раздел 7.5.4, стр. 383):

```
vector<int> v1(10); // ok: прямая инициализация
vector<int> v2 = 10; // ошибка: конструктор, получающий размер, является
                    // явным
void f(vector<int>); // параметр f() инициализируется копией
f(10); // ошибка: нельзя использовать явный конструктор для копирования
        // аргумента
f(vector<int>(10)); // ok: непосредственно создать временный вектор
                   // из int
```

Прямая инициализация вектора `v1` корректна, но на первый взгляд эквивалентная инициализация копией вектора `v2` ошибочна, поскольку конструктор вектора, получающий один параметр размера, является явным. По тем же причинам недопустима инициализация копией вектора `v2` — нельзя неявно использовать явный конструктор при передаче аргумента или возвращении значения из функции. Если нужно использовать явный конструктор, то сделать это следует явно, как в последней строке примера, приведенного выше.

Компилятор может обойти конструктор копий

Во время инициализации копией компилятору можно (но не обязательно) пропустить конструктор копий или перемещения и создать объект непосредственно. Таким образом, код

```
string null_book = "9-999-99999-9"; // инициализация копией
```

компилятор может выполнить так:

```
string null_book("9-999-99999-9"); // компилятор пропускает конструктор
                                   // копий
```

Но даже если компилятор обойдет вызов конструктора копий или перемещения, то он все равно должен существовать и быть доступен (не должен быть закрытым, например) в этой точке программы.

Упражнения раздела 13.1.1

Упражнение 13.1. Что такое конструктор копий? Когда он используется?

Упражнение 13.2. Объясните, почему следующее объявление недопустимо:

```
Sales_data::Sales_data(Sales_data rhs);
```

Упражнение 13.3. Объясните, что происходит при копировании объектов классов `StrBlob` и `StrBlobPtr`?

Упражнение 13.4. Предположим, класс `Point` имеет открытый конструктор копий. Укажите каждый случай использования конструктора копий в этом фрагменте кода:

```
Point global;
Point foo_bar(Point arg)
{
    Point local = arg, *heap = new Point(global);
    *heap = local;
    Point pa[ 4 ] = { local, *heap };
    return *heap;
}
```

Упражнение 13.5. Напишите с учетом следующего эскиза класса конструктор копий, копирующий все переменные-члены. Конструктор должен динамически резервировать новую строку (см. раздел 12.1.2 стр. 584) и копировать объект, на который указывает `ps`, а не сам указатель `ps`.

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
private:
    std::string *ps;
    int i;
};
```



13.1.2. Оператор присвоения копии

Подобно тому, как класс контролирует инициализацию своих объектов, он контролирует также присваивание своих объектов:

```
Sales_data trans, accum;
trans = accum; // использует оператор присвоения копии класса Sales_data
```

Компилятор сам синтезирует оператор присвоения копии, если он не определен в классе явно.

Перегруженный оператор присвоения

Прежде чем перейти к синтезируемому оператору присвоения, необходимо ознакомиться с *перегружкой операторов* (overloaded operator), подробно рассматриваемой в главе 14.

Перегруженные операторы — это функции, имена которых состоят из слова `operator` и символа определяемого оператора. Следовательно, оператор присвоения — это функция `operator=`. Подобно любой другой функции, у функции оператора есть тип возвращаемого значения и список параметров.

Параметрами перегруженного оператора являются его операнды. Некоторые операторы, например присвоение, должны быть определены, как функ-

ции-члены. Когда оператор является функцией-членом, левый операнд связан с неявным параметром `this` (см. раздел 7.1.2, стр. 336). Правый операнд бинарного оператора, такого как присвоение, передается как явный параметр.

Оператор присвоения копии получает аргумент того же типа, что и класс:

```
class Foo {
public:
    Foo& operator=(const Foo&); // оператор присвоения
    // ...
};
```

Для совместимости с оператором присвоения встроенных типов (см. раздел 4.4, стр. 201) операторы присвоения обычно возвращают ссылку на свой левый операнд. Следует также заметить, что библиотека обычно требует от типов, хранимых в контейнере, наличия операторов присвоения, возвращающих ссылку на левый операнд.



Рекомендуем

Операторы присвоения обычно должны возвращать ссылку на свой левый операнд.

Синтезируемый оператор присвоения копии

Подобно конструктору копий, компилятор создает *синтезируемый оператор присвоения копии* (*synthesized assignment operator*) для класса, если в нем не определен собственный. Аналогично конструктору копий, у некоторых классов синтезируемый оператор присвоения копии не подразумевает присвоения (раздел 13.1.6, стр. 644). В противном случае он присваивает значение каждой нестатической переменной-члена правого объекта соответствующей переменной-члену левого объекта с использованием оператора присвоения копии типа этой переменной. Массивы присваиваются поэлементно. Синтезируемый оператор присвоения копии возвращает ссылку на свой левый операнд.

Например, следующий код эквивалентен синтезируемому оператору присвоения копии класса `Sales_data`:

```
// эквивалент синтезируемого оператора присвоения копии
Sales_data&
Sales_data::operator=(const Sales_data &rhs)
{
    bookNo = rhs.bookNo;           // вызов string::operator=
    units_sold = rhs.units_sold; // использует встроенное присвоение int
    revenue = rhs.revenue;        // использует встроенное присвоение double
    return *this;                 // вернуть этот объект
}
```

Упражнения раздела 13.1.2

Упражнение 13.6. Что такое оператор присвоения копии? Когда он используется? Что делает синтезируемый оператор присвоения копии? Когда он синтезируется?

Упражнение 13.7. Что произойдет при присвоении одного объекта класса `StrBlob` другому? Что произойдет при присвоении объектов класса `StrBlobPtr`?

Упражнение 13.8. Напишите оператор присвоения для класса `HasPtr` из упражнения 13.5 раздела 13.1.1 (стр. 634). Подобно конструктору копий, данный оператор присвоения должен копировать объект, на который указывает указатель `ps`.

13.1.3. Деструктор

Действие деструктора противоположно действию конструктора: конструкторы инициализируют нестатические переменные-члены объекта, а также могут выполнять другие действия; деструкторы осуществляют все действия, необходимые для освобождения использованных объектом ресурсов и удаления нестатических переменных-членов объекта.

Деструктор — это функция-член с именем класса, предваряемым тильдой (~). У нее нет ни параметров, ни возвращаемого значения:

```
class Foo {  
public:  
    ~Foo(); // деструктор  
    // ...  
};
```

Поскольку деструктор не получает никаких параметров, он не может быть перегружен. Для каждого класса возможен только один деструктор.

Что делает деструктор

Подобно тому, как конструктор имеет часть инициализации и тело (см. раздел 7.5.1, стр. 372), деструктор имеет тело и часть удаления. В конструкторе переменные-члены инициализируются перед выполнением тела, а инициализация членов осуществляется в порядке их объявления в классе. В деструкторе сначала выполняется тело, а затем происходит удаление членов. Переменные-члены удаляются в порядке, обратном их инициализации.

Тело деструктора осуществляет все операции, которые разработчик класса считает необходимыми выполнить после использования объекта. Как правило, деструктор освобождает ресурсы объекта, зарезервированные на протяжении его существования.

У деструктора нет ничего похожего на список инициализации конструктора для контроля удаления переменных-членов; часть удаления неявна. Происходящее при удалении переменной-члена зависит от его типа. Члены типа класса удаляются за счет выполнения его собственного деструктора. У встроенных типов нет деструкторов, поэтому для удаления членов встроенного типа не делается ничего.



Неявное удаление члена-указателя встроенного типа *не удаляет* объект, на который он указывает.

В отличие от обычных указателей, интеллектуальные указатели (см. раздел 12.1.1, стр. 578) являются классами и имеют деструкторы. Поэтому, в отличие от обычных указателей, члены, являющиеся интеллектуальными указателями, автоматически удаляются на фазе удаления.

Когда происходит вызов деструктора

Деструктор автоматически используется всякий раз, когда удаляется объект его типа.

- Переменные удаляются, когда выходят из области видимости.
- Переменные-члены объекта удаляются при удалении объекта, которому они принадлежат.
- Элементы в контейнере (будь то библиотечный контейнер или массив) удаляются при удалении контейнера.
- Динамически созданные объекты удаляются при применении оператора `delete` к указателю на объект (см. раздел 12.1.2, стр. 586).
- Временные объекты удаляются в конце выражения, в котором они были созданы.

Поскольку деструкторы выполняются автоматически, программы могут резервировать ресурсы и (обычно) не заботиться о том, когда они освобождаются.

Например, следующий фрагмент кода определяет четыре объекта класса `Sales_data`:

```
{ // новая область видимости
    // p и p2 указывают на динамически созданные объекты
    Sales_data *p = new Sales_data;          // p встроенный указатель
    auto p2 = make_shared<Sales_data>();     // p2 - shared_ptr
    Sales_data item(*p);                    // конструктор копий копирует *p в item
    vector<Sales_data> vec;                 // локальный объект
    vec.push_back(*p2);                     // копирует объект, на который указывает p2
    delete p;                               // деструктор вызывается для объекта, на
                                           // который указывает p
} // выход из локальной области видимости; деструктор вызывается
// для item, p2 и vec
```

```
// удаление p2 уменьшает его счетчик пользователей; если значение
// счетчика дойдет до 0, объект освобождается
// удаление вектора vec удалит и его элементы
```

Каждый из этих объектов содержит член типа `string`, который резервирует динамическую память для содержания символов переменной-члена `bookNo`. Но единственная память, которой код должен управлять непосредственно, — это самостоятельно зарезервированный объект. Код непосредственно освобождает только динамически созданный объект, связанный с указателем `p`.

Другие объекты класса `Sales_data` автоматически удаляются при выходе из области видимости. По завершении блока `vec`, `p2` и `item` выходят из области видимости, это означает вызов деструкторов классов `vector`, `shared_ptr` и `Sales_data` для соответствующих объектов. Деструктор класса `vector` удалит элемент, помещенный в вектор `vec`. Деструктор класса `shared_ptr` осуществит декремент счетчика ссылок объекта, на который указывает указатель `p2`. В данном примере этот счетчик достигнет нуля, поэтому деструктор класса `shared_ptr` удалит объект класса `Sales_data`, зарезервированный с использованием указателя `p2`.

Во всех случаях деструктор класса `Sales_data` неявно удаляет переменную-член `bookNo`. Удаление переменной-члена `bookNo` запускает деструктор класса `string`, который освобождает память, используемую для хранения ISBN.



Когда из области видимости выходит ссылка или указатель на объект, деструктор *не выполняется*.

Синтезируемый деструктор

Компилятор определяет *синтезируемый деструктор* (`synthesized destructor`) для любого класса, который не определяет собственный деструктор. Подобно конструкторам копий и операторам присвоения копии, определение для некоторых классов синтезируемого деструктора предотвращает удаление объектов этого типа (раздел 13.1.6, стр. 644). В противном случае у синтезируемого деструктора будет пустое тело.

Например, синтезируемый деструктор класса `Sales_data` эквивалентен следующему:

```
class Sales_data {
public:
    // не делать ничего, кроме удаления переменных-членов,
    // осуществляемого автоматически
    ~Sales_data() { }
    // другие члены как прежде
};
```


Переменные-члены автоматически удаляются после выполнения (пустого) тела деструктора. В частности, деструктор класса `string` будет выполнен для освобождения памяти, используемой переменной-членом `bookNo`.

Важно понять, что само тело деструктора не удаляет переменные-члены непосредственно. Они удаляются в ходе неявной фазы удаления, которая следует за телом деструктора. Тело деструктора выполняется *в дополнение* к удалению членов, осуществляемому в ходе удаления объекта.

Упражнения раздела 13.1.3

Упражнение 13.9. Что такое деструктор? Что делает синтезируемый деструктор? Когда деструктор синтезируется?

Упражнение 13.10. Что произойдет при удалении объекта класса `StrBlob`? А класса `StrBlobPtr`?

Упражнение 13.11. Добавьте деструктор в класс `HasPtr` из предыдущих упражнений.

Упражнение 13.12. Сколько вызовов деструктора происходит в следующем фрагменте кода?

```
bool fcn(const Sales_data *trans, Sales_data accum)
{
    Sales_data item1(*trans), item2(accum);
    return item1.isbn() != item2.isbn();
}
```

Упражнение 13.13. Наилучший способ изучения функций-членов управления копированием и конструкторов — это определить простой класс с этими функциями-членами, каждая из которых выводит свое имя:

```
struct X {
    X() {std::cout << "X()" << std::endl;}
    X(const X&) {std::cout << "X(const X&)" << std::endl;}
};
```

Добавьте в структуру `X` оператор присвоения копии и деструктор, а затем напишите программу, использующую объекты класса `X` различными способами: передайте их как ссылочный и не ссылочный параметры; динамически зарезервируйте их; поместите в контейнеры и т.д. Изучайте вывод, пока не начнете хорошо понимать, когда и почему используется каждая функция-член управления копированием. По мере чтения вывода помните, что компилятор может обойти вызовы конструктора копий.

13.1.4. Правило три/пять

Как уже упоминалось, существуют три базовых функции, контролирующих копирование объектов класса: конструктор копий, оператор присвоения

копии и деструктор. Кроме того, как будет продемонстрировано в разделе 13.6 (стр. 672), по новому стандарту класс может также определить конструктор перемещения и оператор присваивания при перемещении.

Определять все эти функции не обязательно: вполне можно определить один или два из них, не определяя все. Эти функции можно считать модулями. Если нужен один, не обязательно определять их все.

Классы, нуждающиеся в деструкторах, нуждаются в копировании и присвоении

Вот эмпирическое правило, используемое при принятии решения о необходимости определения в классе собственных версий функций-членов управления копированием: сначала следует решить, нужен ли классу деструктор. Зачастую потребность в деструкторе более очевидна, чем потребность в операторе присвоения или конструкторе копий. Если класс нуждается в деструкторе, он почти наверняка нуждается также в конструкторе копий и операторе присвоения копии.

Используемый в упражнениях класс `HasPtr` отлично подойдет для примера (см. раздел 13.1.1, стр. 633). Этот класс резервирует динамическую память в конструкторе. Синтезируемый деструктор не будет удалять указатель-член. Поэтому данный класс должен определить деструктор для освобождения памяти, зарезервированной конструктором.

Хоть это и не очевидно, но согласно эмпирическому правилу класс `HasPtr` нуждается также в конструкторе копий и операторе присвоения копии.

Давайте посмотрим, что было бы, если бы у класса `HasPtr` был деструктор и синтезируемые версии конструктора копий и оператора присвоения копии:

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
    ~HasPtr() { delete ps; }
    // ошибка: HasPtr нуждается в конструкторе копий и операторе
    // присвоения копии
    // другие члены, как прежде
};
```

В этой версии класса зарезервированная в конструкторе память будет освобождена при удалении объекта класса `HasPtr`. К сожалению, здесь есть серьезная ошибка! Данная версия класса использует синтезируемые версии операторов копирования и присвоения. Эти функции копируют указатели-члены, а значит, несколько объектов класса `HasPtr` смогут указывать на ту же область памяти:

```
HasPtr f(HasPtr hp) // HasPtr передан по значению, поэтому он
                    // копируется
```

```
{
    HasPtr ret = hp; // копирует данный HasPtr
    // обработка ret
    return ret; // ret и hp удаляются
}
```

Когда функция `f()` завершает работу, объекты `hp` и `ret` удаляются и деструктор класса `HasPtr` выполняется для каждого из них. Этот деструктор удалит указатель-член и в объекте `ret`, и в объекте `hp`. Но эти объекты содержат одинаковое значение указателя. Код удалит тот же указатель дважды, что является серьезной ошибкой (см. раздел 12.1.2, стр. 589) с непредсказуемыми результатами.

Кроме того, вызывающая сторона функции `f()` может все еще использовать переданный ей объект:

```
HasPtr p("some values");
f(p); // по завершении f() память, на которую указывает p.ps,
      // освобождается
HasPtr q(p); // теперь и p, и q указывают на недопустимую память!
```

Память, на которую указывает указатель `p` (и `q`), больше недопустима. Она была возвращена операционной системе, когда был удален объект `hp` (или `ret`)!



Если класс нуждается в деструкторе, он почти наверняка нуждается также в операторе присвоения копии и конструкторе копий.

Классы, нуждающиеся в копировании, нуждаются также в присвоении, и наоборот

Хотя большинству классов требуется определить все функции-члены управления копированием (или ни один из них), у некоторых классов есть необходимость только в копировании или присвоении объектов, но нет никакой необходимости в деструкторе.

В качестве примера рассмотрим класс, присваивающий каждому своему объекту уникальный последовательный номер. Такому классу нужен конструктор копий для создания нового уникального последовательного номера для создаваемого объекта. Этот конструктор копировал бы все остальные переменные-члены заданного объекта. Класс нуждался бы также в собственном операторе присвоения копии, чтобы избежать присвоения объекту слева последовательного номера. Однако у этого класса не было бы никакой потребности в деструкторе.

Этот пример иллюстрирует второе эмпирическое правило: если класс нуждается в конструкторе копий, то он почти наверняка нуждается в операторе присвоения копии, и наоборот, — если класс нуждается в операторе присвоения, то он почти наверняка нуждается также в конструкторе копий. Од-

нако нужна в конструкторе копий или операторе присвоения копии не означает потребности в деструкторе.

Упражнения раздела 13.1.4

Упражнение 13.14. Предположим, что класс `numbered` имеет стандартный конструктор, создающий уникальный последовательный номер для каждого объекта, который хранится в переменной-члене `mysn`. Класс `numbered` использует синтезируемые функции-члены управления копированием и имеет следующую функцию:

```
void f (numbered s) { cout << s.mysn << endl; }
```

Какой вывод создаст следующий код?

```
numbered a, b = a, c = b;
f(a); f(b); f(c);
```

Упражнение 13.15. Предположим, что у класса `numbered` есть конструктор копий, создающий новый последовательный номер. Изменит ли это вывод вызовов в предыдущем упражнении? Если да, то почему? Какой вывод получится?

Упражнение 13.16. Что если параметром функции `f()` будет `const numbered&`? Это изменяет вывод? Если да, то почему? Какой вывод получится?

Упражнение 13.17. Напишите версии класса `numbered` и функции `f()`, соответствующие трем предыдущим упражнениям, и проверьте правильность предсказания вывода.

13.1.5. Использование спецификатора `= default`

C++ 11 Используя спецификатор `= default`, можно явно указать компилятору на необходимость создать синтезируемые версии функций-членов управления копированием (см. раздел 7.1.4, стр. 345):

```
class Sales_data {
public:
    // управление копированием; версии по умолчанию
    Sales_data() = default;
    Sales_data(const Sales_data&) = default;
    Sales_data& operator=(const Sales_data &);
    ~Sales_data() = default;
    // другие члены как прежде
};
Sales_data& Sales_data::operator=(const Sales_data&) = default;
```

Когда в объявлении функции-члена в теле класса использован спецификатор `= default`, синтезируемая функция неявно становится встраиваемой (как и любая другая функция-член, определенная в теле класса). Если синтезируемая функция-член класса не должна быть встраиваемой функцией, можно до-

бавить часть = default в ее определение, как это было сделано в определении оператора присвоения копии.



Спецификатор = default можно использовать только для тех функций-членов, у которых есть синтезируемая версия (т.е. стандартный конструктор или функция-член управления копированием).

13.1.6. Предотвращение копирования



Большинство классов должно определить (явно или неявно) стандартный конструктор, конструктор копий и оператор присвоения копии.

Хотя большинство классов должно определять (и, как правило, определяет) конструктор копий и оператор присвоения копии, у некоторых классов нет реальной необходимости в этих функциях. В таких случаях класс должен быть определен так, чтобы предотвращать копирование и присвоение. Например, классы `iostream` предотвращают копирование, чтобы не позволять нескольким объектам писать или читать из того же буфера ввода-вывода. Казалось бы, предотвратить копирование можно, и не определяя функции-члены управления копированием. Но эта стратегия не сработает: если класс не определит эти функции, то компилятор синтезирует их сам.

Определение функции как удаленной



По новому стандарту можно предотвратить копирование, определив конструктор копий и оператор присвоения копии как *удаленные функции* (deleted function). Удаленной называется функция, которая была объявлена, но не может использоваться никаким другим способом. Чтобы определить функцию как удаленную, за списком ее параметров следует расположить часть = delete:

```
struct NoCopy {
    NoCopy() = default; // использовать синтезируемый стандартный
                        // конструктор
    NoCopy(const NoCopy&) = delete; // без копирования
    NoCopy &operator=(const NoCopy&) = delete; // без присвоения
    ~NoCopy() = default; // используйте синтезируемый деструктор
    // другие члены
};
```

Часть = delete указывает компилятору (и читателям кода), что эти функции-члены не определяются *преднамеренно*.

В отличие от части = default, часть = delete должна присутствовать в первом объявлении удаленной функции. Это различие согласуется со смыслом данных объявлений. Часть = default влияет только на то, какой код соз-

дает компилятор; следовательно, она необходима, только пока компилятор не создаст код. С другой стороны, компилятор должен знать, что функция удалена, и запретить ее использование другими функциями.

Также в отличие от части = default, часть = delete можно применить для любой функции (= default применима только к стандартному конструктору или функции-члену управления копированием, которую компилятор может синтезировать). Хотя изначально удаленные функции предназначались для подавления функций-членов управления копированием, они иногда применимы также для воздействия на процесс подбора функции.

Деструктор не должен быть удаленной функцией-членом

Следует заметить, что удалять деструктор нельзя. Если его удалить, то не будет никакого способа освободить объект этого типа. Компилятор не позволит определять переменные или создавать временные объекты типа, у которого удален деструктор. Кроме того, нельзя определять переменные или временные объекты класса, обладающего членом, у типа которого удален деструктор. Если у переменной-члена класса удален деструктор, то она не может быть освобождена. Если не может быть удалена переменная-член, не может быть удален и весь объект в целом.

Хотя определить переменные или переменные-члены таких типов нельзя, вполне можно динамически резервировать объекты с удаленным деструктором. Однако впоследствии их нельзя будет освободить:

```
struct NoDtor {
    NoDtor() = default; // использовать синтезируемый стандартный
                        // конструктор
    ~NoDtor() = delete; // нельзя удалять объекты типа NoDtor
};
NoDtor nd; // ошибка: у NoDtor удаленный деструктор
NoDtor *p = new NoDtor(); // ok: но нельзя удалить p
delete p; // ошибка: у NoDtor удаленный деструктор
```



ВНИМАНИЕ

Невозможно определить объект или удалить указатель на динамически созданный объект типа с удаленным деструктором.

Функции-члены управления копированием могут быть синтезированы как удаленные

Как уже упоминалось, если не определены функции-члены управления копированием, компилятор определит их сам. Аналогично, если класс не определяет конструктор, компилятор синтезирует стандартный конструктор для этого класса сам (см. раздел 7.1.4, стр. 342). Для некоторых классов компилятор определяет эти синтезируемые функции-члены как удаленные.

- Синтезируемый деструктор определяется как удаленный, если у класса есть переменная-член, собственный деструктор которой удален или недоступен (например, `private`).
- Синтезируемый конструктор копий определяется как удаленный, если у класса есть переменная-член, собственный деструктор которой удален или недоступен. Он также будет удаленным, если у класса есть переменная-член с удаленным или недоступным деструктором.
- Синтезируемый оператор присвоения копии определяется как удаленный, если у класса есть переменная-член с удаленным или недоступным оператором присвоения копии, либо если у класса есть константный или ссылочный член.
- Синтезируемый стандартный конструктор определяется как удаленный, если у класса есть переменная-член с удаленным или недоступным деструктором; или имеется ссылочный член без внутриклассового инициализатора (см. раздел 2.6.1 стр. 113); или есть константная переменная-член, тип которой не определяет стандартный конструктор явно и не имеет внутриклассового инициализатора.

Короче говоря, эти правила означают, что если у класса есть переменная-член, которая не может быть стандартно создана, скопирована, присвоена или удалена, то соответствующая функция-член класса будет удаленной функцией.

Как ни удивительно, но переменная-член, класс которой имеет удаленный или недоступный деструктор, приводит к определению синтезируемого стандартного конструктора копий удаленным. Основание для этого правила в том, что без него возможно создание объектов, которые невозможно удалить.

Однако в том, что компилятор не будет синтезировать стандартный конструктор для класса со ссылочным или с константным членом, который не может быть создан стандартно, ничего удивительного нет. Нет ничего удивительного и в том, что класс с константным членом не может использовать синтезируемый оператор присвоения копии: в конце концов, этот оператор пытается присвоить значения всем членам классов. Однако присвоить новое значение константному объекту невозможно.

Хотя вполне возможно присвоить новое значение ссылке, это изменит значение объекта, на который она ссылается. Если бы оператор присвоения копии синтезировался для таких классов, то левый операнд продолжил бы ссылаться на тот же объект, что и перед присвоением. Он не ссылался бы на тот же объект, что и правый операнд. Поскольку это поведение вряд ли будет желательным, синтезируемый оператор присвоения копии определяется как удаленный, если у класса есть ссылочный член.

Как будет продемонстрировано в разделах 13.6.2 (стр. 683), 15.7.2 (стр. 787) и 19.6 (стр. 1056), есть и другие аспекты, в связи с которыми функции-члены копирования могут быть определены как удаленные.



Как правило, функции-члены управления копированием синтезируются как удаленные, когда невозможно скопировать, присвоить или удалить член класса.

Закрытые функции управления копированием

До появления нового стандарта классы предотвращали копирование, объявляя свой конструктор копий и оператор присвоения копии как закрытые (private):

```
class PrivateCopy {
    // нет спецификатора доступа; следующие члены являются закрытыми
    // по умолчанию; см. р. 7.2 стр. 350
    // функции управления копированием закрыты, а потому недоступны
    // обычному пользовательскому коду
    PrivateCopy(const PrivateCopy&);
    PrivateCopy &operator=(const PrivateCopy&);
    // другие члены
public:
    PrivateCopy() = default; // использовать синтезируемый стандартный
                           // конструктор
    ~PrivateCopy(); // пользователи могут определять объекты этого типа,
                  // но не копировать их
};
```

Поскольку деструктор является открытым (public), пользователи смогут определять объекты класса PrivateCopy. Но так как конструктор копий и оператор присвоения копии являются закрытыми (private), пользовательский код не сможет копировать такие объекты. Но дружественные классы и члены класса вполне могут создавать копии. Чтобы предотвратить копирование и друзьями, и членами класса, эти функции-члены объявляют закрытыми и не определяют их.

За одним исключением, рассматриваемым в разделе 15.2.1 (стр. 750), вполне допустимо объявлять, но не определять функции-члены (см. раздел 6.1.2, стр. 273). Попытка использования неопределенной функции-члена приведет к отказу во время компоновки. При объявлении (без определения) закрытого конструктора копий можно предотвратить любую попытку скопировать объект класса: пользовательский код, пытающийся сделать копию, будет помечен как ошибочный во время компиляции; попытки копирования в функциях-членах или дружественных классах будут отмечены как ошибка во время редактирования.



Для классов, которые должны предотвратить копирование, следует определить собственный конструктор копий и оператор присвоения копии, используя часть `= delete` вместо объявления их закрытыми.

Упражнения раздела 13.1.6

Упражнение 13.18. Определите класс `Employee`, содержащий имя сотрудника и его уникальный идентификатор. Снабдите класс стандартным конструктором и конструктором, получающим строку, представляющую имя сотрудника. Каждый конструктор должен создавать уникальный идентификатор за счет приращения статической переменной-члена.

Упражнение 13.19. Должен ли класс `Employee` определить собственные версии функций-членов управления копированием? Если да, то почему? Если нет, то тоже почему? Реализуйте все члены управления копированием, в которых, на ваш взгляд, нуждается класс `Employee`.

Упражнение 13.20. Объясните, что происходит при копировании, присвоении и удалении объектов классов `TextQuery` и `QueryResult` из раздела 12.3 (стр. 617).

Упражнение 13.21. Должны ли классы `TextQuery` и `QueryResult` определять собственные версии функций-членов управления копированием? Если да, то почему? Если нет, то почему? Реализуйте функции управления копированием, необходимые, по-вашему, в этих классах.



13.2. Управление копированием и ресурсами

Обычно классы, управляющие ресурсами, расположенными вне его, должны определять функции-члены управления копированием. Как упоминалось в разделе 13.6 (стр. 640), такие классы нуждаются в деструкторах, освобождающих зарезервированные объектом ресурсы. Если класс нуждается в деструкторе, он почти наверняка нуждается также в конструкторе копий и операторе присвоения копии.

Чтобы определить эти функции-члены, сначала следует решить, что будет означать копирование объекта данного типа. Вообще, есть два способа: операцию копирования можно определить так, чтобы класс вел себя, как значение или как указатель.

У классов, которые ведут себя, как значения, есть собственное состояние. При копировании объекта как значения копия и оригинал независимы друг от друга. Внесенные в копию изменения никак не влияют на оригинал, и наоборот.

Классы, действующие как указатели, используют состояние совместно. При копировании объектов таких классов копии и оригиналы используют те же данные. Изменения, внесенные в копии, изменяют также оригинал, и наоборот.

Из использованных ранее библиотечных классов поведением, подобным значениям, обладали классы библиотечных контейнеров и класс `string`. Ничего удивительного, что класс `shared_ptr` демонстрирует поведение, подобное указателю, как и класс `StrBlob` (см. раздел 12.1.1, стр. 582). Типы ввода-вывода и класс `unique_ptr` не допускают ни копирования, ни присвоения, поэтому их поведение не похоже ни на значение, ни на указатель.

Чтобы проиллюстрировать эти два подхода, определим для используемого в упражнениях класса `HasPtr` функции-члены управления копированием. Сначала заставим класс действовать, как значение, а затем повторно реализуем его в версии, ведущей себя, как указатель.

У класса `HasPtr` есть два члена типа `int` и указатель на тип `string`. Обычно классы непосредственно копируют переменные-члены встроенного типа (кроме указателей); такие члены являются значениями, а следовательно, ведут себя обычно, как значения. Происходящее при копировании указателя-члена определяет то, должно ли у такого класса, как `HasPtr`, быть поведение, подобное значению или указателю.

Упражнения раздела 13.2

Упражнение 13.22. Предположим, класс `HasPtr` должен вести себя, как значение. Таким образом, у каждого его объекта должна быть собственная копия строки, на которую указывает объект. Определения функций-членов управления копированием рассматривается в следующем разделе, но уже сейчас известно все необходимое для их реализации. Напишите конструктор копий класса `HasPtr` и оператор присвоения копии прежде, чем продолжите чтение.



13.2.1. Классы, действующие как значения

Для обеспечения поведения, подобного значению, у каждого объекта должна быть собственная копия ресурса, которым управляет класс. Это значит, что у каждого объекта класса `HasPtr` должна быть собственная копия строки, на которую указывает указатель `ps`. Для реализации поведения, подобного значению, классу `HasPtr` нужно следующее.

- Конструктор копий, который копирует строку, а не только указатель.
- Деструктор, освобождающий строку.
- Оператор присвоения копии, освобождающий строку существующего объекта и копирующий ее значение в строку правого операнда.

Вот подобная значению версия класса `HasPtr`:

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
    // у каждого объекта класса HasPtr есть собственный экземпляр строки,
    // на которую указывает указатель ps
    HasPtr(const HasPtr &p):
        ps(new std::string(*p.ps)), i(p.i) { }
    HasPtr& operator=(const HasPtr &);
    ~HasPtr() { delete ps; }
private:
    std::string *ps;
    int i;
};
```

Класс достаточно прост, все, кроме оператора присвоения, определено в теле класса. Первый конструктор получает (необязательный) аргумент типа `string`. Он динамически резервирует собственную копию этой строки и сохраняет ее адрес в указателе `ps`. Конструктор копий также резервирует собственный экземпляр строки. Деструктор освобождает память, зарезервированную ее конструкторами, выполняя оператор `delete` для указателя-члена `ps`.

Подобный значению оператор присвоения копии

Обычно операторы присвоения объединяют действия деструктора и конструктора копий. Подобно деструктору, оператор присвоения освобождает ресурсы левого операнда. Подобно конструктору копий, оператор присвоения копирует данные из правого операнда. Однако критически важно, чтобы эти действия осуществлялись в правильной последовательности, даже если объект присваивается сам себе. Кроме того, по возможности следует писать собственные операторы присвоения так, чтобы они оставляли левый операнд в корректном состоянии, иначе произойдет исключение (см. раздел 5.6.2, стр. 261).

В данном случае можно отработать случай присвоения самому себе (и сделать код устойчивым к исключению), осуществляя сначала копирование правого операнда. После копирования освобождается левый операнд и указатель модифицируется так, чтобы он указывал на вновь зарезервированную строку:

```
HasPtr& HasPtr::operator=(const HasPtr &rhs)
{
    auto newp = new string(*rhs.ps); // скопировать строку
    delete ps; // освободить прежнюю память
    ps = newp; // копировать данные из rhs в этот объект
    i = rhs.i;
    return *this; // вернуть этот объект
}
```

В этом операторе присвоения, безусловно, сначала выполняется работа конструктора: инициализатор `newp` идентичен инициализатору `ps` в конструкторе копий класса `HasPtr`. Затем, как в деструкторе, удаляется строка, на которую в настоящее время указывает указатель `ps`. Остается только скопировать указатель на недавно созданную строку и значение типа `int` из `rhs` в этот объект.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ОПЕРАТОРЫ ПРИСВОЕНИЯ

Создавая оператор присвоения, следует учитывать два момента.

- Операторы присвоения должны работать правильно, если объект присваивается сам себе.
- Большинство операторов присвоения делят работу с деструктором и конструктором копий.

Шаблон разработки оператора присвоения подразумевает сначала копирование правого операнда в локальный временный объект. После копирования вполне безопасно удалить существующие члены левого операнда. Как только левый операнд будет освобожден, копировать данные из временного объекта в переменные-члены левого операнда.

Для иллюстрации важности принятия мер против присвоения самому себе рассмотрим, что случилось бы, выгляди оператор присвоения так:

// НЕПРАВИЛЬНЫЙ способ написания оператора присвоения!

```
HasPtr&
HasPtr::operator=(const HasPtr &rhs)
{
    delete ps; // освобождает строку, на которую указывает этот объект
    // если rhs и *this - тот же объект, произойдет копирование удаленной
    // памяти!
    ps = new string(*(rhs.ps));
    i = rhs.i;
    return *this;
}
```

Если `rhs` и этот объект совпадают, удаление `ps` освободит строку, на которую указывают и `*this`, и `rhs`. При попытке копирования `*(rhs.ps)` в операторе `new` этот указатель указывает уже на недопустимую область памяти. Результат непредсказуем.



Для операторов присвоения критически важно работать правильно, даже если объект присваивается сам себе. Проще всего обеспечить это, скопировав правый операнд перед удалением левого.

Упражнения раздела 13.2.1

Упражнение 13.23. Сравните функции-члены управления копированием, написанные для решения упражнений предыдущего раздела, с кодом, представленным здесь. Убедитесь, что понимаете различия, если таковые вообще есть, между вашим кодом и приведенным в книге.

Упражнение 13.24. Что будет, если в версии класса `HasPtr` данного раздела не определен деструктор? Что если не определен конструктор копий?

Упражнение 13.25. Предположим, необходимо определить версию класса `StrBlob`, действующего как значение. Предположим также, что необходимо продолжить использовать указатель `shared_ptr`, чтобы класс `StrBlobPtr` все еще мог использовать указатель `weak_ptr` для вектора. Переделанный класс будет нуждаться в конструкторе копий и операторе присвоения копии, но не в деструкторе. Объясните, что должны делать конструктор копий и оператор присвоения копий. Объясните, почему класс не нуждается в деструкторе.

Упражнение 13.26. Напишите собственную версию класса `StrBlob`, описанного в предыдущем упражнении.



13.2.2. Определение классов, действующих как указатели

Чтобы класс `HasPtr` действовал как указатель, конструктор копий и оператор присвоения копии должны копировать указатель-член, а не строку, на которую он указывает. Класс все еще будет нуждаться в собственном деструкторе, чтобы освободить память, зарезервированную получающим строку конструктором (см. раздел 13.6, стр. 640). Тем не менее в данном случае деструктор не может односторонне освободить связанную с ним строку. Это можно сделать только тогда, когда исчезнет последний указатель на строку.

Простейший способ заставить класс действовать как указатель — это использовать указатель `shared_ptr` для управления ресурсами в классе. При копировании (или присвоении) копируется (или присваивается) указатель `shared_ptr`. Класс `shared_ptr` сам отслеживает количество пользователей, совместно использующих объект, на который он указывает. Когда пользователей больше нет, класс `shared_ptr` освобождает ресурс.

Но иногда управлять ресурсом следует непосредственно. В таких случаях может пригодиться счетчик ссылок (*reference count*) (см. раздел 12.1.1, стр. 578). Для демонстрации работы счетчика ссылок переопределим класс `HasPtr` так, чтобы обеспечить поведение, подобное указателю, но с использованием собственного счетчика ссылок.

Счетчики ссылок

Счетчик ссылок работает следующим образом.

- В дополнение к инициализации объекта каждый конструктор (кроме конструктора копий) создает счетчик. Этот счетчик отслеживает количество объектов, совместно использующих создаваемые данные. Сразу после создания объект только один, поэтому счетчик инициализируется значением 1.
- Конструктор копий не создает новый счетчик; он копирует переменные-члены переданного ему объекта, включая счетчик. Конструктор копий увеличивает значение этого совместно используемого счетчика, указывая на наличие еще одного пользователя данных этого объекта.
- Деструктор уменьшает значение счетчика, указывая, что стало на одного пользователя совместно используемых данных меньше. Если значение счетчика достигает нуля, деструктор удаляет данные.
- Оператор присвоения копии увеличивает счетчик правого операнда и уменьшает счетчик левого. Если счетчик левого операнда достигает нуля, значит, пользователей больше нет. В данном случае оператор присвоения копии должен удалить данные левого операнда.

Единственное затруднение — это решить, где разместить счетчик ссылок. Счетчик не может быть членом непосредственно класса объекта `HasPtr`. Чтобы убедиться почему, рассмотрим происходящее в следующем примере:

```
HasPtr p1("Hiya!");
HasPtr p2(p1); // p1 и p2 указывают на ту же строку
HasPtr p3(p1); // p1, p2 и p3 указывают на ту же строку
```

Если счетчик ссылок будет храниться в каждом объекте, то как модифицировать его правильно при создании объекта `p3`? Можно увеличить счетчик в объекте `p1` и скопировать счет в `p3`, но как модифицировать счетчик в `p2`?

Один из способов решения этой проблемы в том, чтобы хранить счетчик в динамической памяти. При создании объекта резервируется также и новый счетчик. При копировании или присвоении объекта копируется и указатель на счетчик. Таким образом, и копия, и оригинал укажут на тот же счетчик.

Определение класса счетчика ссылок

Используя счетчик ссылок, можно написать подобную указателю версию класса `HasPtr` следующим образом:

```
class HasPtr {
public:
    // конструктор резервирует новую строку и новый счетчик,
    // устанавливаемый в 1
    HasPtr(const std::string &s = std::string()):
```

```

        ps(new std::string(s)), i(0), use(new std::size_t(1)) {}
// конструктор копий копирует все три переменные-члена и увеличивает
// счетчик
HasPtr(const HasPtr &p):
    ps(p.ps), i(p.i), use(p.use) { ++*use; }
HasPtr& operator=(const HasPtr&);
~HasPtr();
private:
    std::string *ps;
    int i;
    std::size_t *use; // член, отслеживающий количество объектов,
                      // совместно использующих *ps
};

```

Здесь была добавлена новая переменная-член `use`, отслеживающая количество объектов, совместно использующих ту же строку. Получающий строку конструктор резервирует счетчик и инициализирует его значением 1, означающим наличие одного пользователя строкового члена класса этого объекта.

Функции-члены копирования подобного указателю класса используют счетчик ссылок

При копировании или присвоении объектов класса `HasPtr` необходимо, чтобы копия и оригинал указывали на ту же строку. Таким образом, когда копируется объект класса `HasPtr`, копируется сам указатель `ps`, а не строка, на которую он указывает. При копировании увеличивается также счетчик, связанный с этой строкой.

Конструктор копий (определенный в классе) копирует все три члена переданного ему объекта класса `HasPtr`. Этот конструктор увеличивает также значение указателя-члена `use`, означая, что у строки, на которую указывают указатели `ps` и `p.ps`, появился другой пользователь.

Деструктор не может безоговорочно удалить указатель `ps`, поскольку могли бы быть и другие объекты, указывающие на ту же область памяти. Вместо этого деструктор осуществляет декремент счетчика ссылок, означая, что строку совместно используют на один объект меньше. Если счетчик достигает нуля, деструктор освобождает память, на которую указывают указатели `ps` и `use`:

```

HasPtr::~~HasPtr()
{
    if (--*use == 0) { // если счетчик ссылок достиг 0,
        delete ps;    // удалить строку
        delete use;   // и счетчик
    }
}

```

Оператор присвоения копии, как обычно, выполняет действия, общие для конструктора копий и деструктора. Таким образом, оператор присвоения должен увеличить счетчик правого операнда (действие конструктора копий)

и декремент счетчика левого операнда, освобождая по мере необходимости используемую память (действие деструктора).

Кроме того, как обычно, оператор должен учитывать присвоение себя самому. Для этого инкремент счетчика `rhs` осуществляется прежде декремента счетчика в левом операнде.

Таким образом, если оба операнда являются тем же объектом, значение счетчика будет увеличено прежде проверки необходимости удаления указателей `ps` и `use`:

```
HasPtr& HasPtr::operator=(const HasPtr &rhs)
{
    ++rhs.use; // инкремент счетчика пользователей правого операнда
    if (--*use == 0) { // затем декремент счетчика этого объекта
        delete ps; // если никаких других пользователей нет
        delete use; // освободить резервированные члены этого объекта
    }
    ps = rhs.ps; // копировать данные из rhs в этот объект
    i = rhs.i;
    use = rhs.use;
    return *this; // вернуть этот объект
}
```

Упражнения раздела 13.2.2

Упражнение 13.27. Определите собственную версию класса `HasPtr` со счетчиком ссылок.

Упражнение 13.28. С учетом следующих классов реализуйте стандартный конструктор и необходимые функции-члены управления копированием.

```
(a) class TreeNode {
    private:
        std::string value;
        int count;
        TreeNode *left;
        TreeNode *right;
};

(b) class BinStrTree {
    private:
        TreeNode *root;
```

13.3. Функция `swap()`

Кроме функций-членов управления копированием, управляющие ресурсами классы зачастую определяют также функцию `swap()` (см. раздел 9.2.5, стр. 436). Определение функции `swap()` особенно важно для классов, которые планируется использовать с алгоритмами переупорядочивания элементов (см. раздел 10.2.3, стр. 491). Такие алгоритмы вызывают функцию `swap()` всякий раз, когда им нужен обмен двух элементов.

Если класс определяет собственную функцию `swap()`, алгоритм использует именно ее. В противном случае используется функция `swap()`, определенная

библиотекой. Как обычно, хоть мы пока и не знаем, как реализуется функция `swap()`, концептуально несложно заметить, что обмен двух объектов задействует копирование и два присвоения. Например, код обмена двух объектов подобного значению класса `HasPtr` (см. раздел 13.2.1, стр. 648) мог бы выглядеть так:

```
HasPtr temp = v1; // сделать временную копию значения v1
v1 = v2;          // присвоить значение v2 объекту v1
v2 = temp;        // присвоить сохраненное значение v1 объекту v2
```

Этот код дважды копирует строку, которая первоначально принадлежала объекту `v1`: один раз, когда конструктор копий класса `HasPtr` копирует объект `v1` в объект `temp`, и второй раз, когда оператор присвоения присваивает объект `temp` объекту `v2`. Он также копирует строку, которая первоначально принадлежала объекту `v2`, когда объект `v2` присваивается объекту `v1`. Как уже упоминалось, копирование объекта, подобного значению класса `HasPtr`, резервирует новую строку и копирует строку, на которую указывает объект класса `HasPtr`.

В принципе ни одно из этих резервирований памяти не обязательно. Вместо того чтобы резервировать новые копии строки, можно было бы обменять указатели. Таким образом, имело бы смысл обменять два объекта класса `HasPtr` так, чтобы выполнить следующее:

```
string *temp = v1.ps; // создать временную копию указателя в v1.ps
v1.ps = v2.ps;        // присвоить указатель v2.ps указателю v1.ps
v2.ps = temp;         // присвоить сохраненный указатель v1.ps
                      // указателю v2.ps
```

Написание собственной функции swap()

Переопределить стандартное поведение функции `swap()` можно, определив в классе ее собственную версию. Вот типичная реализация функции `swap()`:

```
class HasPtr {
    friend void swap(HasPtr&, HasPtr&);
    // другие члены, как в разделе 13.2.1 (стр. 648)
};
inline
void swap(HasPtr &lhs, HasPtr &rhs)
{
    using std::swap;
    swap(lhs.ps, rhs.ps); // обмен указателями, а не строковыми данными
    swap(lhs.i, rhs.i);   // обмен целочисленными членами
}
```

Все начинается с объявления функции `swap()`, дружественной, чтобы предоставить ей доступ к закрытым переменным-членам класса `HasPtr`. Поскольку функция `swap()` предназначена для оптимизации кода, определим ее как встраиваемую (см. раздел 6.5.2, стр. 312). Тело функции `swap()` вызывает функции `swap()` каждой из переменных-членов заданного объекта. В данном

случае сначала обмениваются указатели, а затем целочисленные члены объектов, связанных с параметрами `rhs` и `lhs`.



В отличие от функций-членов управления копированием, функция `swap()` никогда не бывает обязательной. Однако ее определение может быть важно для оптимизации классов, резервирующих ресурсы.



Функции `swap()` должны вызывать функции `swap()`, а не `std::swap()`

В этом коде есть один важный нюанс: хотя в данном случае это не имеет значения, важно, чтобы функция `swap()` вызвала именно функцию `swap()`, а не `std::swap()`. В классе `HasPtr` переменные-члены имеют встроенные типы. Для встроенных типов нет специализированных версий функции `swap()`. В данном случае она вызывает библиотечную функцию `std::swap()`.

Но если класс имеет член, тип которого обладает собственной специализированной функцией `swap()`, то вызов функции `std::swap()` был бы ошибкой. Предположим, например, что есть другой класс по имени `Foo`, переменная-член `h` которого имеет тип `HasPtr`. Если не написать для класса `Foo` собственную версию функции `swap()`, то будет использована ее библиотечная версия. Как уже упоминалось, библиотечная функция `swap()` осуществляет ненужное копирование строк, управляемых объектами класса `HasPtr`.

Ненужного копирования можно избежать, написав функцию `swap()` для класса `Foo`. Но версию функции `swap()` для класса `Foo` можно написать так:

```
void swap(Foo &lhs, Foo &rhs)
{
    // Ошибка: эта функция использует библиотечную версию
    // функции swap(), а не версию класса HasPtr
    std::swap(lhs.h, rhs.h);
    // обменять другие члены класса Foo
}
```

Этот код нормально компилируется и выполняется. Однако никакого различия в производительности между этим кодом и просто использующим стандартную версию функции `swap()` не будет. Проблема в том, что здесь явно запрошен вызов библиотечной версии функции `swap()`. Однако нужна версия функции не из пространства имен `std`, а определенная в классе `HasPtr`.

Правильный способ написания функции `swap()` приведен ниже.

```
void swap(Foo &lhs, Foo &rhs)
{
    using std::swap;
    swap(lhs.h, rhs.h); // использует функцию swap() класса HasPtr
    // обменять другие члены класса Foo
}
```

Все вызовы функции `swap()` обходятся без квалификаторов. Таким образом, каждый вызов должен выглядеть как `swap()`, а не `std::swap()`. По причинам, рассматриваемым в разделе 16.3 (стр. 876), если есть специфическая для типа версия функции `swap()`, она будет лучшим соответствием, чем таковая из пространства имен `std`. В результате, если у типа есть специфическая версия функции `swap()`, вызов `swap()` будет распознан как относящийся к специфической версии. Если специфической для типа версии нет, то (с учетом объявления `using` для функции `swap()` в области видимости) при вызове `swap()` будет использована версия из пространства имен `std`.

У очень осторожных читателей может возникнуть вопрос: почему объявление `using` функции `swap()` не скрывает объявление функции `swap()` класса `HasPtr` (см. раздел 6.4.1, стр. 307). Причины, по которым работает этот код, объясняются в разделе 18.2.3 (стр. 997).

Использование функции `swap()` в операторах присвоения

Классы, определяющие функцию `swap()`, зачастую используют ее в определении собственного оператора присвоения. Эти операторы используют технологию, известную как *копия и обмен* (*copy and swap*). Она подразумевает обмен левого операнда с копией правого:

```
// обратите внимание: параметр rhs передается по значению. Это значит,
// что конструктор копий класса HasPtr копирует строку в правый
// операнд rhs
HasPtr& HasPtr::operator=(HasPtr rhs)
{
    // обменивает содержимое левого операнда с локальной переменной rhs
    swap(*this, rhs); // теперь rhs указывает на память, которую
                      // использовал этот объект
    return *this; // удаление rhs приводит к удалению указателя в rhs
}
```

В этой версии оператора присвоения параметр не является ссылкой. Вместо этого правый операнд передается по значению. Таким образом, `rhs` — это копия правого операнда. Копирование объекта класса `HasPtr` приводит к резервированию новой копии строки данного объекта.

В теле оператора присвоения вызывается функция `swap()`, обменивающая переменные-члены `rhs` с таковыми в `*this`. Этот вызов помещает указатель, который был в левом операнде, в `rhs`, и указатель, который был в `rhs`, — в `*this`. Таким образом, после вызова функции `swap()` указатель-член в `*this` указывает на недавно зарезервированную строку, являющуюся копией правого операнда.

По завершении оператора присвоения параметр `rhs` удаляется и выполняется деструктор класса `HasPtr`. Этот деструктор освобождает память, на которую теперь указывает `rhs`, освобождая таким образом память, на которую указывал левый операнд.

В этой технологии интересен тот момент, что она автоматически обрабатывает присвоение себя себе и изначально устойчива к исключениям. Копирование правого операнда до изменения левого обрабатывает присвоение себя себе аналогично примененному в нашем первоначальном операторе присвоения (см. раздел 13.2.1, стр. 649). Это обеспечивает устойчивость к исключениям таким же образом, как и в оригинальном определении. Единственный код, способный передать исключение, — это оператор `new` в конструкторе копий. Если исключение произойдет, то это случится прежде, чем имеется левый операнд.



Операторы присвоения, использующие копию и обмен, автоматически устойчивы к исключениям и правильно обрабатывают присвоение себя себе.

Упражнения раздела 13.3

Упражнение 13.29. Объясните, почему вызов функции `swap()` в вызове `swap(HasPtr&, HasPtr&)` не приводит к бесконечной рекурсии.

Упражнение 13.30. Напишите и проверьте функцию `swap()` для подобной значению версии класса `HasPtr`. Снабдите свою функцию `swap()` оператором вывода примечания о ее выполнении.

Упражнение 13.31. Снабдите свой класс оператором `<` и определите вектор объектов класса `HasPtr`. Вставьте в вектор несколько элементов, а затем отсортируйте его (`sort()`). Обратите внимание на то, когда вызывается функция `swap()`.

Упражнение 13.32. Получит ли преимущества подобная указателю версия класса `HasPtr` от определения собственной функции `swap()`? Если да, то в чем это преимущество? Если нет, то почему?

13.4. Пример управления копированием

Несмотря на то что управление копированием обычно необходимо для классов, резервирующих ресурсы, управление ресурсами не единственная причина определения этих функций-членов. У некоторых классов может быть необходимость в учете или других действиях, выполняемых функциями управления копированием.

В качестве примера, нуждающегося в управлении копированием класса для учета, рассмотрим два класса, которые могли бы использоваться в приложении обработки почты. Эти классы, `Message` и `Folder`, представляют соответственно сообщение электронной (или другой) почты и каталог, в котором могло бы находиться это сообщение. Каждое сообщение может находиться в нескольких

папках. Но может существовать только одна копия содержимого любого сообщения. Таким образом, если содержимое сообщения изменится, эти изменения отображаются при просмотре данного сообщения в любой из папок.

Для отслеживания того, какие сообщения в каких папках находятся, каждый объект класса `Message` будет хранить набор указателей на объекты класса `Folder`, в которых они присутствуют, а каждый объект класса `Folder` будет содержать набор указателей на его объекты класса `Message`. Эту конструкцию иллюстрирует рис. 13.1.

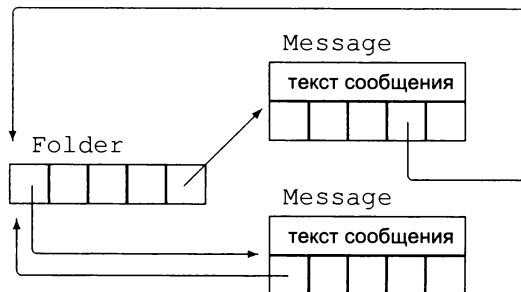


Рис. 13.1. Проект классов `Message` и `Folder`

Класс `Message` будет предоставлять функции `save()` и `remove()` для добавления и удаления сообщений из папки. Для создания нового объекта класса `Message` следует определить содержимое сообщения, но не папку. Чтобы поместить сообщение в определенную папку, следует вызвать функцию `save()`.

После копирования сообщения копия и оригинал будут разными объектами класса `Message`, но оба сообщения должны присутствовать в том же самом наборе папок. Таким образом, копирование сообщения скопирует содержимое и набор указателей на папку. Он должен также добавить указатель на недавно созданный объект класса `Message` к каждому из этих объектов класса `Folder`.

После удаления сообщения объект класса `Message` больше не существует. Поэтому его удаление должно удалять указатели на этот объект класса `Message` из всех объектов класса `Folder`, которые содержали это сообщение.

Когда один объект класса `Message` присваивается другому, содержимое (contents) левого сообщения заменяется таковым правого. Следует также модифицировать набор папок, удалив левый объект класса `Message` из предыдущих объектов класса `Folder` и добавив в них правый.

Глядя на этот список операций, можно заметить, что и деструктор, и оператор присвоения копии должны удалять заданное сообщение из папок, которые указывают на него. Точно так же и конструктор копий, и оператор присвоения копии добавляют объект класса `Message` в заданный список объекта класса `Folder`. Для решения этих задач определим пару закрытых вспомогательных функций.


 Рекомендуем

Оператор присвоения копии зачастую осуществляет ту же работу, которая необходима в конструкторе копий и деструкторе. В таких случаях эти действия обычно помещают в закрытые вспомогательные функции.

Класс `Folder` будет нуждаться в аналогичных функциях-членах управления копированием для добавления и удаления себя из хранящих их объектов класса `Message`.

Проектирование и реализацию класса `Folder` оставим читателю в качестве самостоятельного упражнения, но будем подразумевать, что у него есть функции-члены `addMsg()` и `remMsg()`, выполняющие все действия по добавлению и удалению заданного сообщения из набора сообщений указанной папки.

Класс `Message`

С учетом проекта выше можно написать класс `Message` следующим образом:

```
class Message {
    friend class Folder;
public:
    // папки неявно инициализируются пустым набором
    explicit Message(const std::string &str = ""):
        contents(str) { }
    // функции управления копированием, контролирующие указатели на
    // это сообщение
    Message(const Message&);           // конструктор копий
    Message& operator=(const Message&); // присвоение копии
    ~Message();                       // деструктор
    // добавить/удалить это сообщение из набора сообщений папки
    void save(Folder&);
    void remove(Folder&);
private:
    std::string contents;           // фактический текст сообщения
    std::set<Folder*> folders;      // папки, содержащие это сообщение
    // вспомогательные функции, используемые конструктором копий,
    // оператором присвоения и деструктором
    // добавить это сообщение в папки, на которые указывает параметр
    void add_to_Folders(const Message&);
    // удалить это сообщение из каждой папки в folders
    void remove_from_Folders();
};
```

Класс определяет две переменные-члена: `contents` — для хранения текста сообщения и `folders` — для хранения указателей на объекты класса `Folder`, в которых присутствует данное сообщение. Получающий строку конструктор копирует ее в переменную `contents` и (неявно) инициализирует переменную `folders` пустым набором. Поскольку у этого конструктора есть аргумент по умолчанию, он также является стандартным конструктором класса `Message` (см. раздел 7.5.1, стр. 375).

Функции-члены `save()` и `remove()`

Кроме функций управления копированием, у класса `Message` есть только две открытые функции-члена: `save()`, помещающая сообщение в данную папку, и `remove()`, извлекающая его:

```
void Message::save(Folder &f)
{
    folders.insert(&f); // добавить данную папку в список папок
    f.addMsg(this);     // добавить данное сообщение в набор сообщений
}
void Message::remove(Folder &f)
{
    folders.erase(&f); // удалить данную папку из списка папок
    f.remMsg(this);    // удалить данное сообщение из набора сообщений
}
```

Чтобы сохранить (или удалить) сообщение, требуется модифицировать член `folders` класса `Message`. При сохранении сообщения сохраняется указатель на данный объект класса `Folder`; при удалении сообщения этот указатель удаляется.

Эти функции должны также модифицировать заданный объект класса `Folder`. Модификация этого объекта является задачей, контролируемой классом `Folder` при помощи функций-членов `addMsg()` и `remMsg()`, которые добавляют или удаляют указатель на данный объект класса `Message` соответственно.

Управление копированием класса `Message`

При копировании сообщения копия должна появляться в тех же папках, что и оригинальное сообщение. В результате необходимо перебрать набор указателей класса `Folder`, добавляя указатель на новое сообщение в каждую папку, на которую указывал оригинал сообщения. Для этого и конструктор копий, и оператор присвоения копии должны будут выполнять те же действия, поэтому определим функцию для этой общей работы:

```
// добавить это сообщение в папки, на которые указывает m
void Message::add_to_Folders(const Message &m)
{
    for (auto f : m.folders) // для каждой папки, содержащей m,
        f->addMsg(this);     // добавить указатель на это сообщение
                              // в данную папку
}
```

Здесь происходит вызов функции `addMsg()` для каждого объекта класса `Folder` в `m.folders`. Функция `addMsg()` добавит указатель на этот объект класса `Message` в данный объект класса `Folder`.

Конструктор копий класса `Message` копирует переменные-члены данного объекта:

```

Message::Message(const Message &m):
    contents(m.contents), folders(m.folders)
{
    add_to_Folders(m); // добавить это сообщение в папки, на которые
                      // указывает m
}

```

А также вызывает функцию `add_to_Folders()`, чтобы добавить указатель на недавно созданный объект класса `Message` каждому объекту класса `Folder`, который содержит оригинал сообщения.

Деструктор класса `Message`

При удалении объекта класса `Message` следует удалить это сообщение из папок, которые указывают на него. Это общее действие с оператором присвоения копии, поэтому определим для этого общую функцию:

```

// удалить это сообщение из соответствующих папок
void Message::remove_from_Folders()
{
    for (auto f : folders) // для каждого указателя в folders
        f->remMsg(this); // удалить это сообщение из данной папки
}

```

Реализация функции `remove_from_Folders()` подобна таковой у функции `add_to_Folders()`, за исключением того, что она использует функцию `remMsg()` для удаления текущего сообщения.

При наличии функции `remove_from_Folders()` написать деструктор несложно:

```

Message::~Message()
{
    remove_from_Folders();
}

```

Вызов функции `remove_from_Folders()` гарантирует отсутствие у объектов класса `Folder` указателей на удаленный объект класса `Message`. Компилятор автоматически вызывает деструктор класса `string` для освобождения объекта `contents`, а деструктор класса `set` освобождает память, используемую элементами набора.

Оператор присвоения копии класса `Message`

Как обычно, оператор присвоения и оператор присвоения копии класса `Folder` должны выполнять действия конструктора копий и деструктора. Как всегда, крайне важно структурировать свой код так, чтобы он выполнялся правильно, даже если операнды слева и справа — тот же объект.

В данном случае защита против присвоения самому себе осуществляется за счет удаления указателей на это сообщение из папок левого операнда прежде, чем вставить указатели в папки правого операнда:


```

Message& Message::operator=(const Message &rhs)
{
    // отработать присвоение себе самому, удаляя указатели прежде вставки
    remove_from_Folders(); // обновить существующие папки
    contents = rhs.contents; // копировать содержимое сообщения из rhs
    folders = rhs.folders; // копировать указатели Folder из rhs
    add_to_Folders(rhs); // добавить это сообщение к данным папкам
    return *this;
}

```

Если левый и правый операнды — тот же объект, то у них тот же адрес. Если вызвать функцию `remove_from_Folders()` после вызова функции `add_to_Folders()`, это сообщение будет удалено из всех соответствующих ему папок.

Функция `swap()` класса `Message`

Библиотека определяет версии функции `swap()` для классов `string` и `set` (см. раздел 9.2.5, стр. 436). В результате класс `Message` извлечет пользу из определения собственной версии функции `swap()`. При определении специфической для класса `Message` версии функции `swap()` можно избежать лишних копирований членов `contents` и `folders`.

Но наша функция `swap()` должна также управлять указателями `Folder`, которые указывают на обмениваемые сообщения. После такого вызова, как `swap(m1, m2)`, указатели `Folder`, указывающие на объект `m1`, должны теперь указать на объект `m2`, и наоборот.

Для управления указателями `Folder` осуществляются два прохода по всем элементам `folders`. Первый проход удалит сообщения из соответствующих папок. Затем вызов функции `swap()` совершит обмен переменных-членов. Второй проход по элементам `folders` добавляет указатели на обмениваемые сообщения:

```

void swap(Message &lhs, Message &rhs)
{
    using std::swap; // в данном случае не обязательно, но привычка
                      // хорошая
    // удалить указатели на каждое сообщение из их (оригинальных) папок
    for (auto f: lhs.folders)
        f->remMsg(&lhs);
    for (auto f: rhs.folders)
        f->remMsg(&rhs);
    // обмен наборов указателей contents и folders
    swap(lhs.folders, rhs.folders); // использует swap(set&, set&)
    swap(lhs.contents, rhs.contents); // swap(string&, string&)
    // добавляет указатели на каждое сообщение в их (новые) папки
    for (auto f: lhs.folders)
        f->addMsg(&lhs);
    for (auto f: rhs.folders)
        f->addMsg(&rhs);
}

```

Упражнения раздела 13.4

Упражнение 13.33. Почему параметр функций-членов `save()` и `remove()` класса `Message` имеет тип `Folder&`? Почему этот параметр не определен как `Folder` или `const Folder&`?

Упражнение 13.34. Напишите класс `Message`, как описано в этом разделе.

Упражнение 13.35. Что случилось бы, используй класс `Message` синтезируемые версии функций-членов управления копированием?

Упражнение 13.36. Разработайте и реализуйте соответствующий класс `Folder`. Этот класс должен содержать набор указателей на сообщения в той папке.

Упражнение 13.37. Добавьте в класс `Message` функции-члены удаления и вставки заданного `Folder*` в `folders`. Эти члены аналогичны функциям-членам `addMsg()` и `remMsg()` класса `Folder`.

Упражнение 13.38. Для определения оператора присвоения класса `Message` не использовалась технология копирования и обмена. Почему, по вашему?



13.5. Классы, управляющие динамической памятью

Некоторые классы должны резервировать переменный объем памяти во время выполнения. Такие классы зачастую способны (а если способны, то обычно обязаны) использовать библиотечный контейнер для хранения данных. Например, для хранения своих элементов класс `StrBlob` использует вектор.

Но эта стратегия срабатывает не для каждого класса; некоторые из них должны самостоятельно резервировать память. Обычно такие классы определяют собственные функции-члены управления копированием, чтобы управлять памятью, которую они резервируют.

В качестве примера реализуем упрощенную версию библиотечного класса `vector`. Кроме прочих упрощений, этот класс не будет шаблоном, он сможет хранить только строки. Поэтому назовем этот класс `StrVec`.

Проект класса `StrVec`

Как уже упоминалось, класс `vector` хранит свои элементы в непрерывном хранилище. Для повышения производительности класс `vector` предварительно резервирует хранилище, размер которого превосходит необходимое количество элементов (см. раздел 9.4, стр. 457). Каждая добавляющая элементы функция-член вектора проверяет наличие доступного пространства для следующего элемента. Если это так, элемент размещается в следующей дос-

тупной ячейке. Если места нет, вектор пересоздается: он резервирует новое пространство, перемещает в него существующие элементы, освобождает прежнее пространство и добавляет новый элемент.

Подобную стратегию и будем использовать в классе `StrVec`. Для получения пустой памяти используем класс `allocator` (см. раздел 12.2.2, стр. 613). Поскольку резервируемая классом `allocator` память пуста, используем его функцию-член `construct()` для создания объектов в этом пространстве, когда необходимо добавить новый элемент. Точно так же при удалении элемента используем его функцию-член `destroy()`.

У каждого объекта класса `StrVec` будет три указателя на пространство, используемое для хранения его элементов:

- указатель `elements` на первый элемент в зарезервированной памяти;
- указатель `first_free` на следующий элемент после фактически последнего;
- указатель `cap` на следующий элемент после конца зарезервированной памяти.

Значение этих указателей представлено на рис. 13.2.



Определение класса StrVec

Теперь, сделав набросок реализации, можно определить класс StrVec:

```
// упрощенная реализация стратегии резервирования памяти для подобного
// вектору класса
class StrVec {
public:
    StrVec(): // член allocator инициализируется по умолчанию
        elements(nullptr), first_free(nullptr), cap(nullptr) { }
    StrVec(const StrVec&); // конструктор копий
    StrVec &operator=(const StrVec&); // присвоение копии
    ~StrVec(); // деструктор
    void push_back(const std::string&); // копирует элемент
    size_t size() const { return first_free - elements; }
    size_t capacity() const { return cap - elements; }
    std::string *begin() const { return elements; }
    std::string *end() const { return first_free; }
    // ...
private:
    std::allocator<std::string> alloc; // резервирует элементы
    // используется функциями, которые добавляют элементы в StrVec
    void chk_n_alloc()
        { if (size() == capacity()) reallocate(); }
    // вспомогательные члены, используемые конструктором копий,
    // оператором присвоения и деструктором
    std::pair<std::string*, std::string*> alloc_n_copy
        (const std::string*, const std::string*);
    void free(); // удаляет элементы и освобождает пространство
    void reallocate(); // резервирует больше места и копирует
        // существующие элементы
    std::string *elements; // указатель на первый элемент массива
    std::string *first_free; // указатель на первый свободный
        // элемент массива
    std::string *cap; // указатель на следующий элемент после
        // конца массива
};
```

Тело класса определяет некоторые из своих членов.

- Стандартный конструктор (неявно) инициализирует по умолчанию переменную-член `alloc` и (явно) инициализирует указатели как `nullptr`, означая, что никаких элементов нет.
- Функция-член `size()` возвращает количество фактически используемых элементов, соответствует значению `first_free - elements`.
- Функция-член `capacity()` возвращает количество элементов, которые может содержать объект класса `StrVec`, соответствует значению `cap - elements`.
- Функция-член `chk_n_alloc()` приводит к пересозданию объекта класса `StrVec`, когда больше нет места для добавления следующего элемента. Это происходит при `cap == first_free`.

- Функции-члены `begin()` и `end()` возвращают указатели на первый (т.е. `elements`) и следующий после последнего существующего элемент (т.е. `first_free`) соответственно.

Использование функции-члена `construct()`

Функция `push_back()` вызывает функцию `chk_n_alloc()`, чтобы удостовериться в наличии места для элемента. В случае необходимости функция `chk_n_alloc()` вызовет функцию `reallocate()`. После вызова функции `chk_n_alloc()` функция `push_back()` знает, что место для нового элемента есть. Она запрашивает свой член класса `allocator` создать новый последний элемент:

```
void StrVec::push_back(const string& s)
{
    chk_n_alloc(); // удостовериться в наличии места для другого элемента
    // создать копию s в элементе, на который указывает first_free
    alloc.construct(first_free++, s);
}
```

При использовании класса `allocator` для резервирования памяти следует помнить, что память резервируется *пустой* (см. раздел 12.2.2, стр. 614). Чтобы использовать эту память, следует вызвать функцию `construct()`, которая создаст объект в этой памяти. Первый аргумент функции `construct()` — это указатель на пустое пространство, зарезервированное вызовом функции `allocate()`. Остальные аргументы определяют, какой конструктор использовать при создании объекта в этом пространстве. В данном случае есть только один дополнительный аргумент типа `string`, поэтому этот вызов использует строковый конструктор копий.

Следует заметить, что вызов функции `construct()` осуществляет приращение указателя `first_free`, чтобы он снова указывал на элемент, который предстоит создать. Поскольку используется постфиксный инкремент (см. раздел 4.5, стр. 204), этот вызов создает объект в текущей позиции указателя `first_free`, а инкремент переводит его на следующий пустой элемент.

Функция-член `alloc_n_copy()`

Функция-член `alloc_n_copy()` вызывается при копировании или присвоении объекта класса `StrVec`. У класса `StrVec` будет подобное значению поведение (см. раздел 13.2.1, стр. 648), как у вектора; при копировании или присвоении объекта класса `StrVec` необходимо зарезервировать независимую память и скопировать элементы из оригинала в новый объект класса `StrVec`.

Функция-член `alloc_n_copy()` будет резервировать достаточно места для содержания заданного диапазона элементов, а затем копировать эти элементы во вновь созданное пространство. Эта функция возвращает значение типа `pair` (см. раздел 11.2.3, стр. 545), переменные-члены которого являются указа-

телем на начало нового пространства и следующую позицию после последнего скопированного элемента:

```
pair<string*, string*>
StrVec::alloc_n_copy(const string *b, const string *e)
{
    // резервировать пространство для содержания элементов диапазона
    auto data = alloc.allocate(e - b);
    // инициализировать и вернуть пару, созданную из данных,
    // возвращенных функцией uninitialized_copy()
    return {data, uninitialized_copy(b, e, data)};
}
```

Функция `alloc_n_copy()` вычисляет объем резервируемого пространства, вычитая указатель на первый элемент из указателя на следующий после последнего. Зарезервировав память, функция создает в ней копии заданных элементов.

Копирование осуществляется в операторе `return` при списочной инициализации возвращаемого значения (см. раздел 6.3.2, стр. 296). Указатель-член `first` возвращенной пары указывает на начало зарезервированной памяти; значение для указателя-члена `second` возвращается функцией `uninitialized_copy()` (см. раздел 12.2.2, стр. 616). Это значение будет указателем на следующий элемент после последнего созданного элемента.

Функция-член `free()`

У функции-члена `free()` две обязанности: она должна удалить элементы, а затем освободить пространство, зарезервированное объектом класса `StrVec`. Цикл `for` вызывает функцию `destroy()` класса `allocator`, перебирая элементы в обратном порядке, начиная с последнего существующего элемента и заканчивая первым:

```
void StrVec::free()
{
    // нельзя освободить 0 указателей;
    // если элемент нулевой - не делать ничего
    if (elements) {
        // удалить прежние элементы в обратном порядке
        for (auto p = first_free; p != elements; /* пусто */)
            alloc.destroy(--p);
        alloc.deallocate(elements, cap - elements);
    }
}
```

Функция `destroy()` запускает деструктор класса `string`. Деструктор класса `string` освобождает память, занятую самой строкой.

Как только элементы будут удалены, освобождается пространство, зарезервированное классом `StrVec` при вызове функции `deallocate()`. Указатель, передаваемый функции `deallocate()`, должен быть именно тем, который ранее создал вызов функции `allocate()`. Поэтому перед вызовом функции `deallocate()` сначала проверяется, тот ли это `elements`, а не нулевой.

Функции-члены управления копированием

При наличии функций-членов `alloc_n_copy()` и `free()` функции-члены управления копированием нашего класса очень просты.

```
StrVec::StrVec(const StrVec &s)
{
    // вызов функции alloc_n_copy() для резервирования количества
    // элементов как в s
    auto newdata = alloc_n_copy(s.begin(), s.end());
    elements = newdata.first;
    first_free = cap = newdata.second;
}
```

Конструктор копий вызывает функцию `alloc_n_copy()`, а затем присваивает результат вызова переменным-членам. Возвращаемое значение функции `alloc_n_copy()` является парой указателей. Первый указатель указывает на первый созданный элемент, а второй — на следующий после последнего созданного. Поскольку функция `alloc_n_copy()` резервирует пространство для точно такого количества элементов, которое было задано, указатель `cap` также указывает только на следующий после последнего созданного.

Деструктор вызывает функцию `free()`:

```
StrVec::~StrVec() { free(); }
```

Оператор присвоения копии вызывает функцию `alloc_n_copy()` прежде, чем освободить существующие элементы. Это защищает от копирования в себя самого:

```
StrVec &StrVec::operator=(const StrVec &rhs)
{
    // вызов alloc_n_copy() для резервирования точно такого количества
    // элементов, как в rhs
    auto data = alloc_n_copy(rhs.begin(), rhs.end());
    free();
    elements = data.first;
    first_free = cap = data.second;
    return *this;
}
```

Подобно конструктору копий, оператор присвоения копии использует значения, возвращенные функцией `alloc_n_copy()`, для инициализации своих указателей.



Перемещение, а не копирование элементов при резервировании

Прежде чем приступить к функции `reallocate()`, следует обдумать то, что она должна делать:

- зарезервировать память для нового, большего массива строк;
- заполнить первую часть этого пространства существующими элементами;
- удалить элементы в существующей памяти и освободить ее.

Глядя на этот список, можно заметить, что пересоздание объекта класса `StrVec` влечет за собой копирование каждой строки из прежнего объекта `StrVec` в новый. Даже без подробностей реализации класса `string` известно, что строки ведут себя подобно значению. После копирования новая строка и оригинальная независимы друг от друга. Изменения, внесенные в оригинал, не распространяются на копию, и наоборот.

Поскольку строки действуют, как значения, можно сделать вывод, что у каждой строки должна быть собственная копия составляющих ее символов. Копирование строки должно резервировать память для этих символов, а удаление строки должно освободить используемую ею память.

Копирование строки подразумевает копирование данных, поскольку обычно после копирования строки у нее будет два пользователя. Но когда функция `reallocate()` копирует строки объекта класса `StrVec`, у этих строк будет только один пользователь. Как только копирование элементов из прежнего пространства в новое завершается, исходные строки немедленно удаляются.

Копирование данных этих строк не нужно. Производительность класса `StrVec` будет *значительно выше*, если удастся избежать дополнительных затрат на резервирование и освобождение строк при каждом его пересоздании.

**C++
11**

Конструктор перемещения и функция `std::move()`

Копирования строки можно избежать при помощи двух средств, введенных новой библиотекой. Во-первых, некоторые из библиотечных классов, включая класс `string`, определяют так называемые *конструкторы перемещения* (`move constructor`). Подробности работы конструктора перемещения класса `string` (равно как и все остальные подробности его реализации) не раскрываются. Однако общеизвестно, что конструкторы перемещения обычно “перемещают” ресурсы из заданного объекта в создаваемый. Библиотека гарантирует также то, что “перемещенная” строка останется в допустимом состоянии. В случае класса `string` можно предположить, что у каждого его объекта есть указатель на массив типа `char`. По-видимому, конструктор перемещения класса `string` копирует указатель вместо резервирования нового пространства и копирования символов.

Второе доступное для использования средство — это библиотечная функция `move()`, определенная в заголовке `utility`. Есть два важных момента, которые следует знать о функции `move()`. Во-первых, по причинам, рассматриваемым в разделе 13.6.1 (стр. 673), когда функция `reallocate()` создает строки в новой области памяти, она должна вызвать функцию `move()`, чтобы сообщить о необходимости использования конструктора перемещения класса

string. Если пропустить вызов функции `move()`, то будет использован конструктор копий класса `string`. Во-вторых, по причинам, рассматриваемым в разделе 18.2.3 (стр. 997), объявление `using` (см. раздел 3.1, стр. 124) для функции `move()` обычно не предоставляется. Когда используется функция `move()`, вызывается функция `std::move()`, а не `move()`.

Функция-член `reallocate()`

Используя эту информацию, можно написать собственную функцию `reallocate()`. Сначала вызовем функцию `allocate()`, чтобы зарезервировать новое пространство. При каждом пересоздании объекта класса `StrVec` будем удваивать его емкость. Если вектор `StrVec` пуст, резервируем место для одного элемента:

```
void StrVec::reallocate()
{
    // будем резервировать вдвое больше элементов, чем текущий размер
    auto newcapacity = size() ? 2 * size() : 1;
    // резервировать новую память
    auto newdata = alloc.allocate(newcapacity);
    // переместить данные из прежней памяти в новую
    auto dest = newdata; // указывает на следующую свободную позицию в
                        // новом массиве
    auto elem = elements; // указывает на следующий элемент в старом
                        // массиве
    for (size_t i = 0; i != size(); ++i)
        alloc.construct(dest++, std::move(*elem++));
    free(); // освобождает старое пространство после перемещения
           // элементов
    // обновить структуру данных, чтобы указать на новые элементы
    elements = newdata;
    first_free = dest;
    cap = elements + newcapacity;
}
```

Цикл `for` перебирает существующие элементы и создает соответствующие элементы в новом пространстве. Указатель `dest` используется для указания на область памяти, в которой создается новая строка, а указатель `elem` — для указания на элемент в оригинальном массиве. Для перемещения указателей `dest` и `elem` на следующий элемент этих двух массивов используем постфиксный инкремент.

Второй аргумент в вызове функции `construct()` (т.е. аргумент, определяющий используемый конструктор (см. раздел 12.2.2, стр. 614)) является значением, возвращенным функцией `move()`. Вызов функции `move()` возвращает результат, заставляющий функцию `construct()` использовать конструктор перемещения класса `string`. Поскольку используется конструктор перемещения, управляемая память строки не будет скопирована. Вместо этого каждая создаваемая строка получит в собственность область памяти из строки, на которую указывает указатель `elem`.

После перемещения элементов происходит вызов функции `free()` для удаления прежних элементов и освобождения памяти, которую данный вектор `StrVec` использовал перед вызовом функции `reallocate()`. Сами строки больше не управляют памятью, в которой они располагались; ответственность за их данные была передана элементам нового вектора `StrVec`. Нам неизвестно содержимое строк в памяти прежнего вектора `StrVec`, но нам гарантирована безопасность запуска деструктора класса `string` для этих объектов.

Остается только обновить указатели адресами вновь созданного и инициализированного массива. Указатели `first_free` и `cap` обозначат элемент следующий после последнего созданного и следующий после последнего зарезервированного соответственно.

Упражнения раздела 13.5

Упражнение 13.39. Напишите собственную версию класса `StrVec`, включая функции `reserve()`, `capacity()` (см. раздел 9.4, стр. 458) и `resize()` (см. раздел 9.3.5, стр. 452).

Упражнение 13.40. Добавьте в класс `StrVec` конструктор, получающий аргумент типа `initializer_list<string>`.

Упражнение 13.41. Почему в вызове функции `construct()` в функции `push_back()` был использован постфиксный инкремент? Что случилось бы при использовании префиксного инкремента?

Упражнение 13.42. Проверьте свой класс `StrVec`, используя его в классах `TextQuery` и `QueryResult` (см. раздел 12.3, стр. 617) вместо вектора `vector<string>`.

Упражнение 13.43. Перепишите функцию-член `free()` так, чтобы для удаления элементов вместо цикла `for` использовалась функция `for_each()` и лямбда-выражение (см. раздел 10.3.2, стр. 497). Какую реализацию вы предпочитаете и почему?

Упражнение 13.44. Напишите класс по имени `String`, являющийся упрощенной версией библиотечного класса `string`. У вашего класса должен быть по крайней мере стандартный конструктор и конструктор, получающий указатель на строку в стиле C. Примените для резервирования используемой классом `String` памяти класс `allocator`.

13.6. Перемещение объектов

Одной из главных особенностей нового стандарта является способность перемещать объект, а не копировать. Как упоминалось в разделе 13.1.1 (стр. 631), копирование осуществляется при многих обстоятельствах. При некоторых из

них объект разрушается немедленно после копирования. В этих случаях перемещение объекта вместо копирования способно обеспечить существенное увеличение производительности.

Как было продемонстрировано только что, наш класс `StrVec` — хороший пример лишнего копирования. Во время пересоздания нет никакой необходимости в копировании элементов из старой памяти в новую, лучше перемещение. Вторая причина предпочесть перемещение копированию — это такие классы как `unique_ptr` и классы ввода-вывода. У этих классов есть ресурс (такой как указатель или буфер ввода-вывода), который не допускает совместного использования. Следовательно, объекты этих типов не могут быть скопированы, но могут быть перемещены.

В прежних версиях языка не было непосредственного способа перемещения объекта. Копию приходилось делать, даже если в этом не было никакой потребности. Когда объекты велики или когда они требуют резервирования памяти (например, строки), бесполезное копирование может обойтись очень дорого. Точно так же в предыдущих версиях библиотеки классы хранимых в контейнере объектов должны были допускать копирование. По новому стандарту в контейнерах можно хранить объекты типов, которые не допускают копирования, но могут быть перемещены.



Контейнеры библиотечных типов, классы `string` и `shared_ptr` поддерживают как перемещение, так и копирование. Классы ввода-вывода и класс `unique_ptr` допускают перемещение, но не копирование.



13.6.1. Ссылки на r-значение



Для обеспечения операции пересылки, новый стандарт вводит новый вид ссылок — ссылки на r-значение. Ссылка на r-значение (r-value reference) — это ссылка, которая должна быть связана с r-значением. Ссылку на r-значение получают с использованием символа `&&`, а не `&`. Как будет продемонстрировано далее, у ссылок на r-значение есть важное свойство — они могут быть связаны только с тем объектом, который будет удален. В результате можно “перемещать” ресурсы от ссылки на r-значение в другой объект.

Напомним, что l- и r-значение — свойства выражения (см. раздел 4.1.1, стр. 189). Некоторые выражения возвращают или требуют l-значений; другие возвращают или требуют r-значений. Как правило, выражение l-значения относится к идентификатору объекта, тогда как выражение r-значения — к значению объекта.

Как и любая ссылка, ссылка на r-значение — это только другое имя для объекта. Как известно, нельзя связать обычные ссылки (которые далее будем назы-

вать *ссылками на l-значение* (l-value reference), чтобы отличить их от ссылок на r-значения) с выражениями, требующими преобразования, с литералами и с выражениями, которые возвращают r-значение (см. раздел 2.3.1, стр. 86). У ссылок на r-значение противоположные свойства привязки: можно связать ссылку на r-значение с выражениями, приведенными выше, но нельзя непосредственно связать ссылку на r-значение с l-значением:

```
int i = 42;
int &r = i;    // ok: r ссылается на i
int &&rr = i;   // ошибка: нельзя связать ссылку на r-значение
               // с l-значением
int &r2 = i * 42; // ошибка: i * 42 - это r-значение
const int &r3 = i * 42; // ok: ссылку на константу можно
                       // связать с r-значением
int &&rr2 = i * 42; // ok: связать rr2 с результатом умножения
```

Функции, возвращающие ссылки на l-значение, наряду с присвоением, индексированием, обращением к значению, а также префиксные операторы инкремента и декремента являются примерами выражений, возвращающих l-значения. Ссылку на l-значение можно также связать с результатом любого из этих выражений.

Все функции, возвращающие не ссылочный тип, наряду с арифметическими, реляционными, побитовыми и постфиксными операторами инкремента и декремента возвращают r-значения. С этими выражениями нельзя связать ссылку на l-значение, но можно связать либо константную ссылку на l-значение, либо ссылку на r-значение.

l-значения — устойчивы; r-значения — эфемерны

Глядя на список выражений l- и r-значений, становится понятно, что l- и r-значения существенно отличаются друг от друга: у l-значений есть постоянное состояние, тогда как r-значения, литералы и временные объекты создаются лишь в ходе вычисления выражений.

Поскольку ссылки на r-значение могут быть связаны только с временным объектом, известно, что:

- упомянутый объект будет удален,
- у этого объекта не может быть других пользователей.

Совместно эти факты означают, что использующий ссылку на r-значение код способен получать ресурсы от объекта, на который ссылается ссылка.



Ссылки на r-значение ссылаются на объекты, которые будут вскоре удалены. Следовательно, можно “захватить” состояние объекта, связанного со ссылкой на r-значение.

Переменные являются l-значениями

Хотя мы редко думаем об этом, переменная — это выражение с одним операндом и без оператора. Подобно любому другому выражению, переменная как выражение имеет свойства l- и r-значения. Переменные как выражения — это l-значения. Удивительно, но как следствие невозможно связать ссылку на r-значение с переменной, определенной как тип ссылки на r-значение:

```
int &&rr1 = 42; // ok: литералы - это r-значения
int &&rr2 = rr1; // ошибка: выражение rr1 - это l-значение!
```

С учетом предыдущего наблюдения, согласно которому r-значения представляют эфемерные объекты, нет ничего удивительного в том, что переменная представляет собой l-значение. В конце концов, переменная сохраняется, пока не выйдет из области видимости.



Переменная — это l-значение; нельзя непосредственно связать ссылку на r-значение с переменной, даже если эта переменная была определена как тип ссылки на r-значение.

C++
11

Библиотечная функция `move()`

Хотя нельзя непосредственно связать ссылку на r-значение с l-значением, можно явно привести l-значение к соответствующему типу ссылки на r-значение. Вызов новой библиотечной функции `move()`, определенной в заголовке `utility`, позволяет также получить ссылку на r-значение, привязанную к l-значению. Для возвращения ссылки на r-значение на данный объект функция `move()` использует средства, описываемые в разделе 16.2.6 (стр. 867):

```
int &&rr3 = std::move(rr1); // ok
```

Вызов функции `move()` указывает компилятору, что имеющееся l-значение следует рассматривать как r-значение. Следует помнить, что приведенный выше вызов функции `move()` обещает не использовать `rr1` ни для чего, кроме присвоения или удаления. После вызова функции `move()` нельзя сделать никаких предположений о значении уже перемещенного объекта.



Перемещенный объект можно удалить, а можно присвоить ему новое значение, но значение уже перемещенного объекта использовать нельзя.

Как уже упоминалось, для использования большинства имен из библиотеки, включая функцию `move()` (см. раздел 13.5, стр. 671), не нужно предоставлять объявление `using` (см. раздел 3.1, стр. 124). Произойдет вызов функции `std::move()`, а не `move()`. Причины этого рассматриваются в разделе 18.2.3 (стр. 997).



Код, применяющий функцию `move()`, должен использовать синтаксис `std::move()`, а не `move()`. Это позволит избежать возможных конфликтов имен.

Упражнения раздела 13.6.1

Упражнение 13.45. В чем разница между ссылкой на *r*-значение и ссылкой на *l*-значение.

Упражнение 13.46. Какой вид ссылки может быть связан со следующими инициализаторами?

```
int f();
vector<int> vi(100);
int? r1 = f();
int? r2 = vi[0];
int? r3 = r1;
int? r4 = vi[0] * f();
```

Упражнение 13.47. Снабдите конструктором копий и оператором присвоения копии класса `String` из упражнения 13.44 раздела 13.5 (стр. 672), функции которого выводят сообщения при каждом вызове.

Упражнение 13.48. Определите вектор `vector<String>` и вызовите для него функцию `push_back()` несколько раз. Запустите программу и посмотрите, как часто копируются строки.



13.6.2. Конструктор перемещения и присваивание при перемещении

Подобно классу `string` (и другим библиотечным классам), наши собственные классы могут извлечь пользу из способности перемещения ресурсов вместо копирования. Чтобы позволить собственным типам операции перемещения, следует определить конструктор перемещения и оператор присваивания при перемещении. Эти члены подобны соответствующим функциям копирования, но они захватывают ресурсы заданного объекта, а не копируют их.



Как и у конструктора копий, у конструктора перемещения есть начальный параметр, являющийся ссылкой на тип класса. В отличие от конструктора копий, ссылочный параметр конструктора перемещения является ссылкой на *r*-значение. Подобно конструктору копий, у всех дополнительных параметров должны быть аргументы по умолчанию.

Кроме перемещения ресурсов, конструктор перемещения должен гарантировать такое состояние перемещенного объекта, при котором его удаление будет безопасно. В частности, сразу после перемещения ресурса оригинальный объект больше не должен указывать на перемещенный ресурс, ответственность за него принимает вновь созданный объект.

В качестве примера определим конструктор перемещения для класса `StrVec`, чтобы перемещать, а не копировать элементы из одного объекта класса `StrVec` в другой:

```
StrVec::StrVec(StrVec &&s) noexcept // перемещение не будет передавать
                                   // исключений
    // инициализаторы членов получают ресурсы из s
    : elements(s.elements), first_free(s.first_free), cap(s.cap)
{
    // оставить s в состоянии, при котором запуск деструктора безопасен
    s.elements = s.first_free = s.cap = nullptr;
}
```

Оператор `noexcept` (уведомляющий о том, что конструктор не передает исключений) описан ниже, а пока рассмотрим, что делает этот конструктор.

В отличие от конструктора копий, конструктор перемещения не резервирует новую память; он получает ее от заданного объекта класса `StrVec`. Получив область памяти от своего аргумента, тело конструктора присваивает указателям заданного объекта значение `nullptr`. После перемещения оригинальный объект продолжает существовать. В конечном счете оригинальный объект будет удален, а значит, будет выполнен его деструктор. Деструктор класса `StrVec` вызывает функцию `deallocate()` для указателя `first_free`. Если забыть изменить указатель `s.first_free`, то удаление оригинального объекта освободит область памяти, которая была только что передана.



Операции перемещения, библиотечные контейнеры и исключения

Поскольку операция перемещения выполняется при “захвате” ресурсов, она обычно не резервирует ресурсы. В результате операции перемещения обычно не передают исключений. Когда создается функция перемещения, неспособная передавать исключения, об этом факте следует сообщить библиотеке. Как будет описано вскоре, если библиотека не знает, что конструктор перемещения не будет передавать исключений, она предпримет дополнительные меры по отработке возможности передачи исключения при перемещении объекта этого класса.



Один из способов сообщить об этом библиотеке — определить оператор `noexcept` в конструкторе. Введенный новым стандартом оператор `noexcept` подробно рассматривается в разделе 18.1.4 (стр. 974), а пока достаточно знать, что он позволяет уведомить, что функция не будет передавать исключений. Оператор `noexcept` указывают после списка параметров функции. В конструкторе его располагают между списком параметров и символом `:`, начинающим список инициализации конструктора:

```
class StrVec {
public:
```

```

    StrVec(StrVec&&) noexcept; // конструктор перемещения
    // другие члены, как прежде
};
StrVec::StrVec(StrVec &&s) noexcept : /* инициализаторы членов */
{ /* тело конструктора */ }

```

Оператор `noexcept` следует объявить и в заголовке класса, и в определении, если оно расположено вне класса.



Конструкторы перемещения и операторы присваивания при перемещении, которые не могут передавать исключения, должны быть отмечены как `noexcept`.

Понимание того, почему необходим оператор `noexcept`, может помочь углубить понимание того, как библиотека взаимодействует с объектами написанных вами типов. В основе требования указывать, что функция перемещения не будет передавать исключения, лежат два взаимосвязанных факта: во-первых, хотя функции перемещения обычно не передают исключений, им это разрешено. Во-вторых, библиотечные контейнеры предоставляют гарантии относительно того, что они будут делать в случае исключения. Например, класс `vector` гарантирует, что, если исключение произойдет при вызове функции `push_back()`, сам вектор останется неизменным.

Теперь рассмотрим происходящее в функции `push_back()`. Подобно соответствующей функции класса `StrVec` (см. раздел 13.5, стр. 667), функция `push_back()` класса `vector` могла бы потребовать пересоздания вектора. При пересоздании вектор перемещает элементы из прежней своей области памяти в новую, как в функции `reallocate()` (см. раздел 13.5, стр. 671).

Как только что упоминалось, перемещение объекта обычно изменяет состояние оригинального объекта. Если пересоздание использует конструктор перемещения и этот конструктор передает исключение после перемещения некоторых, но не всех элементов, возникает проблема. Перемещенные элементы в прежнем пространстве были бы изменены, а незаполненные элементы в новом пространстве еще не будут созданы. В данном случае класс `vector` не удовлетворял бы требованию оставаться неизменным при исключении.

С другой стороны, если класс `vector` использует конструктор копий, то при исключении он может легко удовлетворить это требование. В данном случае, пока элементы создаются в новой памяти, прежние элементы остаются неизменными. Если происходит исключение, вектор может освободить зарезервированное пространство (оно могло бы и не быть успешно зарезервировано) и прекратить операцию. Элементы оригинального вектора все еще существуют.

Во избежание этой проблемы класс `vector` должен использовать во время пересоздания конструктор копий вместо конструктора перемещения, *если только не известно*, что конструктор перемещения типа элемента не может передать исключение. Если необходимо, чтобы объекты типа были перемеще-

ны, а не скопированы при таких обстоятельствах, как пересоздание вектора, то следует явно указать библиотеке, что использовать конструктор перемещения безопасно. Для этого конструктор перемещения (и оператора присваивания при перемещении) следует отметить как `noexcept`.

Оператор присваивания при перемещении

Оператор присваивания при перемещении делает то же, что и деструктор с конструктором перемещения. Подобно конструктору перемещения, если оператор присваивания при перемещении не будет передавать исключений, то его следует объявить как `noexcept`. Подобно оператору присвоения копии, оператор присваивания при перемещении должен принять меры против присвоения себя себе:

```
StrVec &StrVec::operator=(StrVec &&rhs) noexcept
{
    // прямая проверка на присвоение себя себе
    if (this != &rhs) {
        free(); // освободить существующие элементы
        elements = rhs.elements; // получить ресурсы от rhs
        first_free = rhs.first_free;
        cap = rhs.cap;
        // оставить rhs в удаляемом состоянии
        rhs.elements = rhs.first_free = rhs.cap = nullptr;
    }
    return *this;
}
```

В данном случае осуществляется прямая проверка совпадения адресов в указателях `rhs` и `this`. Если это так, то правый и левый операнды относятся к тому же объекту, и делать ничего не надо. В противном случае следует освободить память, которую использовал левый операнд, а затем принять память от заданного объекта. Как и в конструкторе перемещения, указателю `rhs` присваивается значение `nullptr`.

Может показаться удивительным, что мы потрудились проверить присвоение себя самому. В конце концов, присваивание при перемещении требует для правого операнда `r`-значения. Проверка осуществляется потому, что то `r`-значение могло быть результатом вызова функции `move()`. Подобно любому другому оператору присвоения, крайне важно не освобождать ресурсы левого операнда прежде, чем использовать (возможно, те же) ресурсы правого операнда.



Исходный объект перемещения должен быть в удаляемом состоянии

Перемещение объекта не удаляет его оригинал: иногда после завершения операции перемещения оригинальный объект следует удалить. Поэтому, создавая функцию перемещения, следует гарантировать, что после перемещения

оригинальный объект будет находиться в состоянии, допускающем запуск деструктора. Функция перемещения класса `StrVec` выполняет это требование и присваивает указателям-членам оригинального объекта значение `nullptr`.

Кроме гарантии безопасного удаления оригинального объекта, функции перемещения должны оставлять объект в допустимом состоянии. Обычно допустимым считается тот объект, которому может быть безопасно присвоено новое значение или который может быть использован другими способами, не зависящими от его текущего значения. С другой стороны, у функций перемещения нет никаких требований относительно значения, которое остается в оригинальном объекте. Таким образом, программы никогда не должны зависеть от значения оригинального объекта после перемещения.

Например, при перемещении объекта библиотечного класса `string` или контейнера известно, что оригинальный объект перемещения остается допустимым. В результате для оригинальных объектов перемещения можно выполнять такие функции, как `empty()` или `size()`. Однако предсказать результат их выполнения затруднительно. Логично было бы ожидать, что оригинальный объект перемещения будет пуст, но это не гарантируется.

Функции перемещения класса `StrVec` оставляют оригинальный объект перемещения в том же состоянии, в котором он находился бы после инициализации по умолчанию. Поэтому все функции класса `StrVec` продолжают выполняться с его объектом точно так же, как с любым другим инициализированным по умолчанию объектом класса `StrVec`. Другие классы, с более сложной внутренней структурой, могут вести себя по-другому.

**ВНИМАНИЕ**

После операции перемещения "оригинальный объект" должен остаться корректным, допускающим удаление объектом, но для пользователей его значение непредсказуемо.

Синтезируемые функции перемещения

Подобно конструктору копий и оператору присвоения копии, компилятор способен сам синтезировать конструктор перемещения и оператор присваивания при перемещении. Однако условия, при которых он синтезирует функции перемещения, весьма отличаются от тех, при которых он синтезирует функции копирования.

Помните, что если не объявить собственный конструктор копий или оператор присвоения копии, компилятор *всегда* синтезирует их сам (см. раздел 13.1.1, стр. 631 и раздел 13.1.2, стр. 634). Функции копирования определяются или как функции почленного копирования либо присвоения объекта, или как удаленные функции.

В отличие от функций копирования, для некоторых классов компилятор не синтезирует функции перемещения *вообще*. В частности, если класс определяет собственный конструктор копий, оператор присвоения копии или деструктор, конструктор перемещения и оператор присваивания при перемещении не синтезируются. В результате у некоторых классов нет конструктора перемещения или оператора присваивания при перемещении. Как будет продемонстрировано вскоре, когда у класса нет функции перемещения, вместо него в результате обычного подбора функции будет использована соответствующая функция копирования.

Компилятор синтезирует конструктор перемещения или оператор присваивания при перемещении, *только если* класс не определяет ни одной из собственных функций-членов управления копированием и если каждая нестатическая переменная-член класса может быть перемещена. Компилятор может перемещать члены встроенного типа, а также члены типа класса, если у него есть соответствующая функция-член перемещения:

```
// компилятор синтезирует функции перемещения для X и hasX
struct X {
    int i; // встроенные типы могут быть перемещены
    std::string s; // string определяет собственные функции перемещения
};
struct hasX {
    X mem; // для X синтезированы функции перемещения
};
X x, x2 = std::move(x); // использует синтезируемый конструктор
                        // перемещения
hasX hx, hx2 = std::move(hx); // использует синтезируемый конструктор
                              // перемещения
```



ВНИМАНИЕ

Компилятор синтезирует конструктор перемещения и оператор присваивания при перемещении, только если класс не определяет ни одной из собственных функций-членов управления копированием и только если все переменные-члены могут быть созданы перемещением и присвоены при перемещении соответственно.

- В отличие от функций копирования, функции перемещения никогда не определяются неявно как удаленные. Но если явно запросить компилятор создать функцию перемещения, применив `= default` (см. раздел 7.1.4, стр. 345), но компилятор окажется неспособен переместить все члены, то функция перемещения будет определена как удаленная. Важное исключение из правила, согласно которому синтезируемая функция перемещения определяется как удаленная, подобно таковому для функций копирования (см. раздел 13.1.6, стр. 644).

- В отличие от конструктора копий, конструктор перемещения определяется как удаленный, если у класса есть член, определяющий собственный конструктор копий, но не определяющий конструктор перемещения, или если у класса есть член, который не определяет собственные функции копирования и для которого компилятор неспособен синтезировать конструктор перемещения. То же относится к присваиванию при перемещении.
- Конструктор перемещения и оператор присваивания при перемещении определяются как удаленные, если у класса есть член, собственный конструктор перемещения которого или оператор присваивания при перемещении которого удален или недоступен.
- Как и конструктор копий, конструктор перемещения определяется как удаленный, если деструктор удален или недоступен.
- Как и оператор присвоения копии, оператор присваивания при перемещении определяется как удаленный, если у класса есть константный или ссылочный член.

Предположим, например, что в классе `Y` определен собственный конструктор копий, но не определен собственный конструктор перемещения:

```
// класс Y определяет собственный конструктор копий, но не конструктор
// перемещения
struct hasY {
    hasY() = default;
    hasY(hasY&&) = default;
    Y mem; // hasY будет иметь удаленный конструктор перемещения
};
hasY hy, hy2 = std::move(hy); // ошибка: конструктор перемещения удален
```

Компилятор может скопировать объекты типа `Y`, но не может переместить их. Класс `hasY` явно запросил конструктор перемещения, который компилятор не способен создать. Следовательно, класс `hasY` получит удаленный конструктор перемещения. Если бы у класса `hasY` отсутствовало объявление конструктора перемещения, то компилятор не синтезировал бы конструктор перемещения вообще. Функции перемещения не синтезируются, если в противном случае они были определены как удаленные.

И последнее взаимоотношение между функциями перемещения и синтезируемыми функциями-членами управления копированием: тот факт, определяет ли класс собственные функции перемещения, влияет на то, как синтезируются функции копирования. Если класс определит любой конструктор перемещения и (или) оператор присваивания при перемещении, то синтезируемый конструктор копий и оператор присвоения копии для этого класса будут определены как удаленные.



Классы, определяющие конструктор перемещения или оператор присваивания при перемещении, должны также определять собственные функции копирования. В противном случае эти функции-члены по умолчанию удаляются.

R-значения перемещаются, а l-значения копируются...

Когда у класса есть и конструктор перемещения и конструктор копий, компилятор использует обычный подбор функции, чтобы выяснить, какой из конструкторов использовать (см. раздел 6.4, стр. 305). С присвоением точно так же. Например, в классе `StrVec` версия копирования получает ссылку на `const StrVec`. В результате она применима к любому типу, допускающему приведение к классу `StrVec`. Версия перемещения получает `StrVec&&` и применима только к аргументам r-значениям (неконстантным):

```
StrVec v1, v2;
v1 = v2;                // v2 - l-значение; присвоение копии
StrVec getVec(istream &); // getVec возвращает r-значение
v2 = getVec(cin);        // getVec(cin) - r-значение;
                        // присвоение перемещения
```

В первом случае оператору присвоения передается объект `v2`. Его типом является `StrVec`, а выражение `v2` является l-значением. Версия присвоения при перемещении не является подходящей (см. раздел 6.6, стр. 317), поскольку нельзя неявно связать ссылку на r-значение с l-значением. Следовательно, в этом случае используется оператор присвоения копии.

Во втором случае присваивается результат вызова функции `getVec()`, — это r-значение. Теперь подходящими являются оба оператора присвоения — результат вызова функции `getVec()` можно связать с любым параметром оператора. Вызов оператора присвоения копии требует преобразования в константу, в то время как `StrVec&&` обеспечивает точное соответствие. Следовательно, второе присвоение использует оператор присваивания при перемещении.

...но r-значения копируются, если нет конструктора перемещения

Что если класс имеет конструктор копий, но не определяет конструктор перемещения? В данном случае компилятор не будет синтезировать конструктор перемещения. Это значит, что у класса есть конструктор копий, но нет конструктора перемещения. Если у класса нет конструктора перемещения, подбор функции гарантирует, что объекты этого типа будут копироваться, даже при попытке перемещения их вызовом функции `move()`:

```
class Foo {
public:
    Foo() = default;
```

```

Foo(const Foo&); // конструктор копий
// другие члены, но Foo не определяет конструктор перемещения
};
Foo x;
Foo y(x); // конструктор копий; x - это 1-значение
Foo z(std::move(x)); // конструктор копий, поскольку конструктора
// перемещения нет

```

Вызов функции `move(x)` при инициализации объекта `z` возвращает указатель `Foo&&`, привязанный к объекту `x`. Конструктор копий для класса `Foo` является подходящим, поскольку вполне допустимо преобразовать `Foo&&` в `const Foo&`. Таким образом, инициализация объекта `z` использует конструктор копий класса `Foo`.

Следует заметить, что использование конструктора копий вместо конструктора перемещения почти безусловно безопасно (то же справедливо и для оператора присвоения). Обычно конструктор копий отвечает требованиям соответствующего конструктора перемещения: он копирует заданный объект и оставляет оригинальный объект в допустимом состоянии. Конструктор копий, напротив, не будет изменять значение оригинального объекта.



Если у класса будет пригодный конструктор копий и не будет конструктора перемещения, то объекты будут перемещены конструктором копий. То же справедливо для оператора присвоения копии и присвоения при перемещении.

Операторы присвоения копии и обмена и перемещение

Версия класса `HasPtr`, определявшая *оператор присвоения копии и обмена* (copy-and-swap assignment operator) (см. раздел 13.3, стр. 657), — хорошая иллюстрация взаимодействия механизма подбора функции и функций перемещения. Если в этот класс добавить конструктор перемещения, то фактически будет получен также оператор присваивания при перемещении:

```

class HasPtr {
public:
    // добавлен конструктор перемещения
    HasPtr(HasPtr &p) noexcept : ps(p.ps), i(p.i) {p.ps = 0;}
    // оператор присвоения - и оператор перемещения, и присвоения копии
    HasPtr& operator=(HasPtr rhs)
        { swap(*this, rhs); return *this; }
    // другие члены как в р. 13.2.1 (стр. 648)
};

```

В этой версии класса добавлен конструктор перемещения, получающий значения из своего аргумента. Тело конструктора обнуляет указатель-член данного объекта класса `HasPtr`, чтобы гарантировать безопасное удаление оригинального объекта перемещения. Эта функция не делает ничего, она не может передать исключение, поэтому отметим ее как `noexcept` (см. раздел 13.6.2, стр. 677).

Теперь рассмотрим оператор присвоения. У него есть не ссылочный параметр, а значит, этот параметр инициализируется копией (см. раздел 13.1.1, стр. 631). В зависимости от типа аргумента инициализация копией использует либо конструктор копий, либо конструктор перемещения; l-значения копируются, а r-значения перемещаются. В результате этот оператор однократного присвоения действует и как присвоение копии, и как присваивание при перемещении.

Предположим, например, что объекты `hp` и `hp2` являются объектами класса `HasPtr`:

```
hp = hp2; // hp2 - l-значение; для копирования hp2 используется
          // конструктор копий
hp = std::move(hp2); // hp2 перемещает конструктор перемещения
```

В первом случае присвоения правый операнд — l-значение, поэтому конструктор перемещения не подходит. Для инициализации `rhs` будет использоваться конструктор копий. Он будет резервировать новую строку и копировать ту строку, на которую указывает `hp2`.

Во втором случае присвоения вызывается функция `std::move()` для связывания ссылки на r-значение с объектом `hp2`. В данном случае подходят и конструктор копий, и конструктор перемещения. Но поскольку аргумент — это ссылка на r-значение, точное соответствие обеспечит конструктор перемещения. Конструктор перемещения копирует указатель из объекта `hp2` и не резервирует память.

Независимо от того, использовался ли конструктор копии или перемещения, тело оператора присвоения обменивает содержимое двух своих операндов. Обмен объектов класса `HasPtr` приводит к обмену указателями-членами и переменными-членами (типа `int`) этих двух объектов. После вызова функции `swap()` правый операнд будет содержать указатель на строку, который ранее принадлежал левому. При выходе `rhs` из области видимости эта строка будет удалена.

СОВЕТ. ОБНОВЛЕННОЕ ПРАВИЛО ТРЕХ

Все пять функций-членов управления копированием можно считать единым блоком: если класс определяет любую из этих функций, он должен обычно определять их все. Как уже упоминалось, для правильной работы некоторые классы *должны* определять конструктор копий, оператор присвоения копии и деструктор (см. раздел 13.6, стр. 640). Как правило, у таких классов есть ресурс, который должны копировать функции-члены копирования. Обычно копирование ресурса влечет за собой некоторые дополнительные затраты. Классы, определяющие конструктор перемещения и оператор присваивания при перемещении, могут избежать этих затрат в тех обстоятельствах, где копия не обязательна.

Функции перемещения для класса `Message`

Классы, определяющие собственный конструктор копий и оператор присвоения копии, обычно определяют и функции перемещения. Например, наши классы `Message` и `Folder` (см. раздел 13.4, стр. 658), должны определять функции перемещения. При определении функций перемещения класс `Message` может использовать функции перемещения классов `string` и `set`, чтобы избежать дополнительных затрат при копировании членов `contents` и `folders`.

Но в дополнение к перемещению члена `folders` следует также обновить каждый объект класса `Folder`, указывавший на оригинал объекта класса `Message`. Следует также удалить указатели на прежний объект класса `Message` и добавить указатели на новый.

И конструктор перемещения, и оператор присваивания при перемещении должны обновлять указатели `Folder`, поэтому начнем с определения функций для выполнения этих действий:

```
// переместить указатели Folder из m в данное Message
void Message::move_Folders(Message *m)
{
    folders = std::move(m->folders); // использует присвоение перемещения
                                     // класса set
    for (auto f : folders) { // для каждого Folder
        f->remMsg(m);        // удалить старый Message из Folder
        f->addMsg(this);     // добавить этот Message в этот Folder
    }
    m->folders.clear(); // гарантировать безопасное удаление m
}
```

Функция начинает работу с перемещения набора `folders`. При вызове функции `move()` используется оператор присвоения при перемещении класса `set`, а не его оператор присвоения копии. Если пропустить вызов функции `move()`, код все равно будет работать, но осуществляя ненужное копирование. Затем функция перебирает папки, удаляя указатель на оригинал сообщения и добавляя указатель на новое сообщение.

Следует заметить, что вставка элемента в набор может привести к передаче исключения, поскольку добавление элемента на контейнер требует резервирования памяти, вполне может быть передано исключение `bad_alloc` (см. раздел 12.1.2, стр. 586). Таким образом, в отличие от функций перемещения классов `HasPtr` и `StrVec`, конструктор перемещения и операторы присваивания при перемещении класса `Message` могли бы передать исключения, поэтому не будем отмечать их как `noexcept` (см. раздел 13.6.2, стр. 677).

Функция заканчивается вызовом функции `clear()` объекта `m.folders`. Известно, что после перемещения объект `m.folders` вполне допустим, но его содержимое непредсказуемо. Поскольку деструктор класса `Message` перебирает набор `folders`, необходимо убедиться, что набор пуст.

Конструктор перемещения класса `Message` вызывает функцию `move()`, чтобы переместить содержимое и инициализировать по умолчанию свой член `folders`:

```
Message::Message(Message &m) : contents(std::move(m.contents))
{
    move_Folders(&m); // переместить folders и обновить указатели Folder
}
```

В теле конструктора происходит вызов функции `move_Folders()`, чтобы удалить указатели на `m` и вставить указатели на данное сообщение.

Оператор присваивания при перемещении непосредственно проверяет случай присвоения себя себе:

```
Message& Message::operator=(Message &&rhs)
{
    if (this != &rhs) { // прямая проверка присвоения себя себе
        remove_from_Folders();
        contents = std::move(rhs.contents); // присвоение при перемещении
        move_Folders(&rhs); // сбросить папки, чтобы указывать на это
                               // сообщение
    }
    return *this;
}
```

Подобно любым операторам присвоения, оператор присваивания при перемещении должен удалить прежние данные левого операнда. В данном случае удаление левого операнда требует удаления указателей на это сообщение из существующих папок, что и делает вызов функции `remove_from_Folders()`. После удаления из папок происходит вызов функции `move()`, чтобы переместить `contents` из объекта `rhs` в `this`. Остается только вызвать функцию `move_Folders()`, чтобы модифицировать указатели `Folder`.

Итераторы перемещения

Функция `reallocate()` класса `StrVec` (см. раздел 13.5, стр. 671) использовала вызов функции `construct()` в цикле `for` для копирования элементов из прежней памяти в новую. Альтернативой циклу был бы просто вызов функции `uninitialized_copy()` для создания нового пространства в памяти. Однако функция `uninitialized_copy()` делает именно то, о чем говорит ее имя: она копирует элементы. Нет никакой аналогичной библиотечной функции для перемещения объектов в пустую память.

C++
11

Вместо нее новая библиотека определяет адаптер *итератора перемещения* (move iterator) (см. раздел 10.4, стр. 514). Итератор перемещения адаптирует переданный ему итератор, изменяя поведение его оператора обращения к значению. Обычно оператор обращения к значению итератора возвращает ссылку на l-значение элемента. В отличие от других итераторов, оператор обращения к значению итератора перемещения возвращает ссылку на r-значение.

Обычный итератор преобразуется в итератор перемещения при вызове библиотечной функции `make_move_iterator()`, которая получает итератор и возвращает итератор перемещения.

Все остальные функции первоначального итератора работают, как обычно. Поскольку эти итераторы поддерживают обычные функции итераторов, пару итераторов перемещения вполне можно передать алгоритму. В частности, итераторы перемещения можно передать алгоритму `uninitialized_copy()`:

```
void StrVec::reallocate()
{
    // зарезервировать вдвое больше пространства, чем для текущего
    // количества элементов
    auto newcapacity = size() ? 2 * size() : 1;
    auto first = alloc.allocate(newcapacity);
    // переместить элементы
    auto last = uninitialized_copy(make_move_iterator(begin()),
                                  make_move_iterator(end()),
                                  first);

    free(); // освободить прежнее пространство
    elements = first; // обновить указатели
    first_free = last;
    cap = elements + newcapacity;
}
```

Алгоритм `uninitialized_copy()` вызывает функцию `construct()` для каждого элемента исходной последовательности, чтобы скопировать элемент по назначению. Для выбора элемента из исходной последовательности данный алгоритм использует оператор обращения к значению итератора. Поскольку был передан итератор перемещения, оператор обращения к значению возвращает ссылку на r-значение. Это означает, что функция `construct()` будет использовать для создания элементов конструктор перемещения.

Следует заметить, что стандартная библиотека не дает гарантий применимости всех алгоритмов с итераторами перемещения. Так как перемещение объекта способно удалить оригинал, итераторы перемещения следует передавать алгоритмам, только тогда, когда вы *уверены*, что алгоритм не будет обращаться к элементам после того, как он присвоил этот элемент или передал его пользовательской функции.

СОВЕТ. НЕ СЛИШКОМ СПЕШИТЕ С ПЕРЕМЕЩЕНИЕМ

Поскольку состояние оригинального объекта перемещения неопределенно, вызов для него функции `std::move()` — опасная операция. Когда происходит вызов функции `move()`, следует быть абсолютно уверенным в том, что у оригинального объекта перемещения не может быть никаких других пользователей.

Взвешенно использованная в коде класса, функция `move()` способна обеспечить существенный выигрыш в производительности. Небрежное ее использование в обычном пользовательском коде (в отличие от кода реализации класса), вероятней всего, приведет к загадочным и трудно обнаруживаемым ошибкам, а не к повышению производительности приложения.

Рекомендуем

За пределами кода реализации класса, такого как конструкторы перемещения или операторы присваивания при перемещении, используйте функцию `std::move()` только при абсолютной *уверенности* в необходимости перемещения и в том, что перемещение гарантированно будет безопасным.

Упражнения раздела 13.6.2

Упражнение 13.49. Добавьте конструктор перемещения и оператор присваивания при перемещении в классы `StrVec`, `String` и `Message`.

Упражнение 13.50. Снабдите функции перемещения класса `String` операторами вывода и снова запустите программу из упражнения 13.48 раздела 13.6.1 (стр. 676), в котором использовался вектор `vector<String>`, и посмотрите, когда теперь удастся избежать копирования.

Упражнение 13.51. Хотя указатель `unique_ptr` не может быть скопирован, в разделе 12.1.5 (стр. 601) была написана функция `clone()`, которая возвратила указатель `unique_ptr` по значению. Объясните, почему эта функция допустима и как она работает.

Упражнение 13.52. Объясните подробно, что происходит при присвоении объектов класса `HasPtr` на стр. 685. В частности, опишите шаг за шагом, что происходит со значениями `hp`, `hp2` и параметром `rhs` в операторе присвоения класса `HasPtr`.

Упражнение 13.53. С точки зрения низкоуровневой эффективности оператор присвоения класса `HasPtr` не идеален. Объясните почему. Реализуйте для класса `HasPtr` оператор присвоения копии и присваивания при перемещении и сравните действия, выполняемые в новом операторе присваивания при перемещении, с версией копии и обмена.

Упражнение 13.54. Что бы случилось, если бы мы определили оператор присваивания при перемещении для класса `HasPtr`, но не изменили оператор копии и обмена? Напишите код для проверки вашего ответа.



13.6.3. Ссылки на r-значение и функции-члены

Все функции-члены, кроме конструкторов и операторов присвоения, могут извлечь пользу из предоставления версии копирования и перемещения. Такие функции-члены с поддержкой перемещения обычно используют ту же схему параметров, что и конструктор копий/перемещения и операторы присвоения, — одна версия получает ссылку на константное l-значение, а вторая — ссылку на не константное r-значение.

Например, библиотечные контейнеры, определяющие функцию `push_back()`, предоставляют две версии: параметр одной является ссылкой на r-значение, а другой — ссылкой на константное l-значение. С учетом того, что `X` является типом элемента, эти функции контейнера определяются так:

```
void push_back(const X&); // копирование: привязка к любому X
void push_back(X&&);      // перемещение: привязка только к изменяемым
                          // r-значениям типа X
```

Первой версии функции `push_back()` можно передать любой объект, который может быть приведен к типу `X`. Эта версия копирует данные своего параметра. Второй версии можно передать только r-значение, которое не является константой. Эта версия точнее и лучше соответствует неконстантным r-значениям и будет выполнена при передаче поддающегося изменению r-значения (см. раздел 13.6.2, стр. 683). Эта версия способна захватить ресурсы своего параметра.

Обычно нет никакой необходимости определять версии функций получающих `const X&&` или просто `X&`. Обычно ссылку на r-значение передают при необходимости “захватить” аргумент. Для этого аргумент не должен быть константой. Точно так же копирование объекта не должно изменять скопированный объект. В результате обычно нет никакой необходимости определять версию, получающую простой параметр `X&`.



У перегруженных функций, различающих перемещение и копирование параметра, обычно есть одна версия, получающая параметр типа `const T&`, и вторая, получающая параметр типа `T&&`.

В качестве более конкретного примера придадим классу `StrVec` вторую версию функции `push_back()`:

```
class StrVec {
public:
```

```

    void push_back(const std::string&); // копирует элемент
    void push_back(std::string&&);    // перемещает элемент
// другие члены как прежде
};
// неизменно с оригинальной версии в разделе 13.5 (стр. 667)
void StrVec::push_back(const string& s)
{
    chk_n_alloc(); // удостовериться в наличии места для другого элемента
    // создать копию s в элементе, на который указывает first_free
    alloc.construct(first_free++, s);
}
void StrVec::push_back(string &s)
{
    chk_n_alloc(); // пересоздает StrVec при необходимости
    alloc.construct(first_free++, std::move(s));
}

```

Эти функции-члены почти идентичны. Различие в том, что версия ссылки на r-значение функции `push_back()` вызывает функцию `move()`, чтобы передать этот параметр функции `construct()`. Как уже упоминалось, функция `construct()` использует тип своего второго и последующих аргументов для определения используемого конструктора. Поскольку функция `move()` возвращает ссылку на r-значение, аргумент функции `construct()` будет иметь тип `string&&`. Поэтому для создания нового последнего элемента будет использован конструктор перемещения класса `string`.

Когда вызывается функция `push_back()`, тип аргумента определяет, копируется ли новый элемент в контейнер или перемещается:

```

StrVec vec; // пустой StrVec
string s = "some string or another";
vec.push_back(s); // вызов push_back(const string&)
vec.push_back("done"); // вызов push_back(string&&)

```

Эти вызовы различаются тем, является ли аргумент l-значением (`s`) или r-значением (временная строка, созданная из слова "done"). Вызовы распознаются соответственно.

Ссылки на l-значения, r-значения и функции-члены

Обычно функцию-член объекта можно вызвать независимо от того, является ли этот объект l- или r-значением. Например:

```

string s1 = "a value", s2 = "another";
auto n = (s1 + s2).find('a');

```

Здесь происходит вызов функции-члена `find()` (см. раздел 9.5.3, стр. 467) для r-значения класса `string`, полученного при конкатенации двух строк. Иногда такой способ применения может удивить:

```

s1 + s2 = "wow!";

```

Здесь *r*-значению присваивается результат конкатенации двух строк.

До нового стандарта не было никакого способа предотвратить подобное применение. Для обеспечения совместимости с прежней версией библиотечные классы продолжают поддерживать присвоение *r*-значению; в собственных классах такое может понадобиться предотвратить. В таком случае левый операнд (т.е. объект, на который указывает указатель *this*) обязан быть *l*-значением.

C++ 11 Свойство *l*- или *r*-значения указателя *this* задают таким же образом, как и константность функции-члена (см. раздел 7.1.2, стр. 337): помещая *квалификатор ссылки* (reference qualifier) после списка параметров:

```
class Foo {
public:
    Foo &operator=(const Foo&) & ; // возможно присвоение только
                                   // изменяемым l-значениям
    // другие члены класса Foo
};
Foo &Foo::operator=(const Foo &rhs) &
{
    // сделать все необходимое для присвоения rhs этому объекту
    return *this;
}
```

Квалификаторы ссылки *&* или *&&* означают, что указатель *this* может указывать на *r*- или *l*-значение соответственно. Подобно спецификатору *const*, квалификатор ссылки может быть применен только к (нестатической) функции-члену и должен присутствовать как в объявлении, так и в определении функции.

Функцию, квалифицированную символом *&*, можно применить только к *l*-значению, а функцию, квалифицированную символом *&&*, — только к *r*-значению:

```
Foo &retFoo(); // возвращает ссылку; вызов retFoo() является l-значением
Foo retVal(); // возвращает значение; вызов retVal() - r-значение
Foo i, j;      // i и j - это l-значения
i = j;         // ok: i - это l-значение
retFoo() = j;  // ok: retFoo() возвращает l-значение
retVal() = j;  // ошибка: retVal() возвращает r-значение
i = retVal();  // ok: вполне можно передать r-значение как правый
               // операнд присвоения
```

Функция может быть квалифицирована и ссылкой, и константой. В таких случаях квалификатор ссылки должен следовать за спецификатором *const*:

```
class Foo {
public:
    Foo someMem() & const; // ошибка: первым должен быть спецификатор const
    Foo anotherMem() const & ; // ok: спецификатор const расположен первым
};
```

Перегрузка и ссылочные функции

Подобно тому, как можно перегрузить функцию-член на основании константности параметра (см. раздел 7.3.2, стр. 359), ее можно перегрузить на основании квалификатора ссылки. Кроме того, функцию можно перегрузить на основании квалификатора ссылки и константности. В качестве примера придадим классу `Foo` член типа `vector` и функцию `sorted()`, возвращающую копию объекта класса `Foo`, в котором сортируется вектор:

```
class Foo {
public:
    Foo sorted() &&;           // применимо к изменяемым r-значениям
    Foo sorted() const &;      // применимо к любому объекту класса Foo
    // другие члены класса Foo
private:
    vector<int> data;
};
// этот объект - r-значение, поэтому его можно сортировать на месте
Foo Foo::sorted() &&
{
    sort(data.begin(), data.end());
    return *this;
}
// этот объект либо константа, либо l-значение; так или иначе, его нельзя
// сортировать на месте
Foo Foo::sorted() const & {
    Foo ret(*this);           // создает копию
    sort(ret.data.begin(), ret.data.end()); // сортирует копию
    return ret;               // возвращает копию
}
```

При выполнении функции `sorted()` для `r`-значения вполне безопасно сортировать вектор-член `data` непосредственно. Объект является `r`-значением, а это означает, что у него нет никаких других пользователей, поэтому данный объект можно изменить непосредственно. При выполнении функции `sorted()` для константного `r`- или `l`-значения изменить этот объект нельзя, поэтому перед сортировкой вектор-член `data` необходимо скопировать.

Поиск перегруженной функции использует свойство `l`-значение/`r`-значение объекта, вызвавшего функцию `sorted()` для определения используемой версии:

```
retVal().sorted(); // retVal() - это r-value, вызов Foo::sorted() &&
retFoo().sorted(); // retFoo() - это l-value, вызов Foo::sorted() const &
```

При определении константных функций-членов можно определить две версии, отличающиеся только тем, что одна имеет квалификатор `const`, а другая нет. Для ссылочной квалификации функций ничего подобного по умолчанию нет. При определении двух или более функций-членов с тем же именем и тем же списком параметров следует предоставить квалификатор ссылки для всех или ни для одной из этих функций:

```

class Foo {
public:
    Foo sorted() &&;
    Foo sorted() const; // ошибка: должен быть квалификатор ссылки
    // Comp - псевдоним для типа функции (см. р. 6.7, стр. 324)
    // он применим для сравнения целочисленных значений
    using Comp = bool(const int&, const int&);
    Foo sorted(Comp*); // ok: другой список параметров
    Foo sorted(Comp*) const; // ok: ни одна из версий не квалифицирована
                                // как ссылка
};

```

Здесь объявление константной версии функции `sorted()` без параметров является ошибкой. Есть вторая версия функции `sorted()` без параметров, и у нее есть квалификатор ссылки, поэтому у константной версии этой функции также должен быть квалификатор ссылки. С другой стороны, те версии функции `sorted()`, которые получают указатель на функцию сравнения, прекрасно работают, поскольку ни у одной из функций нет спецификатора.



Если у функции-члена есть квалификатор ссылки, то у всех версий этой функции-члена с тем же списком параметров должны быть квалификаторы ссылки.

Упражнения раздела 13.6.3

Упражнение 13.55. Добавьте в класс `StrBlob` функцию `push_back()` в версии ссылки на `r`-значение.

Упражнение 13.56. Что бы было при таком определении функции `sorted()`:

```

Foo Foo::sorted() const & {
    Foo ret(*this);
    return ret.sorted();
}

```

Упражнение 13.57. Что если бы функция `sorted()` была определена так:

```

Foo Foo::sorted() const & { return Foo(*this).sorted(); }

```

Упражнение 13.58. Напишите версию класса `Foo` с операторами вывода в функциях `sorted()`, чтобы проверить свои ответы на два предыдущих упражнения.

Резюме

Каждый класс контролирует происходящее при копировании, перемещении, присвоении и удалении объектов его типа. Эти действия определяют специальные функции-члены: конструктор копий, конструктор перемещения, оператор присвоения копии, оператор присваивания при перемещении и деструктор.

Конструктор перемещения и оператор присваивания при перемещении (обычно неконстантный) получают ссылку на *г*-значение; версии оператора копирования (обычно константные) получают обычную ссылку на *l*-значение.

Если класс не объявит ни одну из этих функций, то компилятор определит их автоматически. Если они не определены как удаленные, эти функции-члены инициализирует, перемещают, присваивают и удаляют объект, обрабатывая каждую нестатическую переменную-член по очереди. Синтезируемая функция делает то, что соответствует типу элемента для перемещения, копирования, присвоения и удаления этого элемента.

Классы, резервирующие память или другие ресурсы, почти всегда требуют, чтобы класс определил функции-члены управления копированием для управления зарезервированным ресурсом. Если класс нуждается в деструкторе, то он почти наверняка должен определить конструкторы перемещения и копирования, а также операторы перемещения и присвоения копии.

Термины

Деструктор (destructor). Специальная функция-член, освобождающая занятую объектом память, когда он выходит из области видимости или удаляется. Компилятор автоматически удаляет каждый член класса. При удалении переменных-членов типа класса используются их собственные деструкторы, а при удалении переменных-членов встроенного или составного типа конструктор ничего не делает. В частности, объект, на который указывает указатель-член класса, автоматически не удаляется деструктором.

Инициализация копией (copy initialization). Форма инициализации с использованием оператора `=` и предоставления инициализатора для создаваемого объекта. Используется также при передаче и возвращении объекта по значению, при инициализации массива или агрегатного класса. Инициализация копией использует конструктор копий или конструктор перемещения, в зависимости от того, является ли инициализатор *l*- или *г*-значением.

Итератор перемещения (move iterator). Адаптер, позволяющий создать итератор, обращение к значению которого возвращает ссылку на *г*-значение.

Квалификатор ссылки (reference qualifier). Символ, обычно указывающий, что нестатическая функция-член может быть вызвана для *l*- или *г*-значения. Спецификатор `&` или `&&` следует за списком параметров или спецификатором `const`, если он есть. Функция с квалификатором `&` может быть вызвана только для *l*-значений, а функция с квалификатором `&&` — только для *г*-значений.

Конструктор копий (copy constructor). Конструктор, который инициализирует новый объект как копию другого объекта того же типа. При передаче объекта в функцию или из функции конструктор копий применяется неявно. Если конструктор копий не определен явно, компилятор синтезирует его самостоятельно.

Конструктор перемещения (move constructor). Конструктор, получающий ссылку на *г*-значение своего типа. Как правило, конструктор перемещения перемещает данные

своего параметра во вновь созданный объект. После перемещения запуск деструктора для правого операнда должен быть безопасен.

Копирование и обмен (copy and swap). Техника написания операторов присвоения за счет копирования правого операнда, сопровождаемого вызовом функции `swap()`, обменивающей копию с левым операндом.

Оператор присваивания при перемещении (move-assignment operator). Версия оператора присвоения, получающая ссылку r-значения на ее тип. Как правило, оператор присваивания при перемещении перемещает данные из правого операнда в левый. После присвоения запуск деструктора для правого операнда должен быть безопасен.

Оператор присвоения копии (copy-assignment operator). Версия оператора присвоения, получающая объект того же типа, что и у нее. Обычно оператор присвоения копии имеет параметр, являющийся ссылкой на константу, и возвращает ссылку на свой объект. Компилятор сам синтезирует оператор присвоения копии, если класс не предоставляет его явно.

Перегруженный оператор (overloaded operator). Функция, переопределяющая один из операторов для работы с операндами данного класса. В этой главе описано определение лишь оператора присвоения, а более подробно перегрузка операторов рассматривается в главе 14.

Почленное копирование и присвоение (memberwise copy/assign). Так работают синтезируемые конструкторы копирования и перемещения, а также операторы присваивания при перемещении и копии. Перебирая все нестатические переменные-члены по очереди, синтезируемый конструктор копий или перемещения инициализирует каждую из них, копируя или при перемещая соответствующее значение из заданного объекта; оператор присваивания при перемещении и копии присваивают при перемещении или копируют каждую переменную-член правого объекта в левый. Инициализация и присвоение переменных-членов встроенного или составного типа осуществляются непосредственно, а членов типа класса — с использованием соответствующего конструктора перемещения или копирования либо оператора присвоения копии или присваивания при перемещении.

Синтезируемые конструкторы копирования и перемещения (synthesized copy/move constructor). Версии конструкторов копирования и перемещения, синтезируемые компилятором для классов, которые не определяют соответствующие конструкторы явно. Если они не определены как удаленные функции, синтезируемые конструкторы копирования и перемещения почленно инициализируют новый объект, копируя или перемещая члены из заданного объекта.

Синтезируемый деструктор (synthesized destructor). Версия деструктора, создаваемая (синтезируемая) компилятором для классов, в которых он не определен явно. Тело синтезируемого деструктора пусто.

Синтезируемый оператор присвоения (synthesized assignment operator). Версия оператора присвоения, создаваемого (синтезируемого) компилятором для классов, у которых он не определен явно. Если он не определен как удаленная функция, синтезируемый оператор присвоения почленно присваивает (перемещает) правый операнд левому.

Ссылка на l-значение (l-value reference). Ссылка, которая может быть связана с l-значением.

Ссылка на r-значение (r-value reference). Ссылка на объект, который будет удален.

- Счетчик ссылок** (reference count). Программное средство, обычно используемое в членах управления копированием. Счетчик ссылок отслеживает количество объектов, совместно использующих некую сущность. Конструкторы (кроме конструкторов копирования и перемещения) устанавливают счетчик ссылок в 1. Каждый раз, когда создается новая копия, значение счетчика увеличивается. Когда объект удаляется, значение счетчика уменьшается. Оператор присвоения и деструктор проверяют, не достиг ли декремент счетчика ссылок нуля, и если это так, то они удаляют объект.
- Удаленная функция** (deleted function). Функция, которая не может быть использована. Для удаления функции в ее объявление включают часть `= delete`. Обычно удаленные функции используют для запрета компилятору синтезировать операторы копирования и (или) перемещения для класса.
- Управление копированием** (copy control). Специальные функции-члены, которые определяют действия, осуществляемые при копировании, присвоении и удалении объектов класса. Если эти функции не определены в классе явно, компилятор синтезирует их самостоятельно.
- Функция** `move()`. Библиотечная функция, обычно используемая для связи ссылки r-значения с l-значением. Вызов функции `move()` неявно обещает, что объект не будет использован для перемещения, кроме его удаления или присвоения нового значения.

